UNIVERSIDAD SIMÓN BOLÍVAR
DECANATO DE ESTUDIOS PROFESIONALES
COORDINACIÓN DE INGENIERÍA ELECTRÓNICA

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA IOT CON MICROCONTROLADORES PARA EL MONITOREO REMOTO DE EQUIPOS FISCALES

Por
Lorena Del Valle Rojas Chams El Dein

INFORME DE PASANTÍA
Presentado ante la ilustre Universidad Simón Bolívar
como requisito requisito parcial para optar al título de
Ingeniería Electrónica

Sartenejas, Septiembre de 2026

L. ROJAS
2026

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA IOT CON MICROCONTROLADORES PARA EL MONITOREO REMOTO DE EQUIPOS FISCALES

USB
INGENIERÍA
ELECTRÓNICA

UNIVERSIDAD SIMÓN BOLÍVAR
DECANATO DE ESTUDIOS PROFESIONALES
COORDINACIÓN DE INGENIERÍA ELECTRÓNICA

DISEÑO E IMPLEMENTACIÓN DE UN SISTEMA IOT CON MICROCONTROLADORES PARA EL MONITOREO REMOTO DE EQUIPOS FISCALES

Por
Lorena Del Valle Rojas Chams El Dein

Realizado con la asesoría de:

Tutor Académico: Prof. Miguel Díaz
Tutor Industrial: Ing. Pablo Arguinzones y Ing. Luis Muñoz (The Factory HKA)

INFORME DE PASANTÍA
Presentado ante la ilustre Universidad Simón Bolívar
como requisito parcial para optar al título de
Ingeniería Electrónica

Sartenejas, Septiembre de 2026

Página reservada para el acta de evaluación

DEDICATORIA

A la mujer y pilar fundamental detrás de cada uno de mis logros: mi mamá.

Fue ella quien, desde muy joven, sembró en mí el sueño de estudiar en la Universidad Simón Bolívar, y quien día a día, sin descanso, me impulsó a permanecer en esta casa de estudios. Sin importar lo difíciles o distantes que se volvieran las circunstancias, sus palabras eran una constante: insistir, persistir y nunca desistir.

Por su entrega, su apoyo incondicional y sus ánimos incansables, este logro es por y para ella.

Te amo, mamá.

AGRADECIMIENTOS

Primeramente, y por encima de todo, a Dios Todopoderoso, por ser mi guía y refugio constante. En los momentos en que el camino se volvía tan empinado que me faltaban las fuerzas para cargar con mis responsabilidades, bastaba leer un versículo de la Biblia para recobrar el aliento, renovar la fe y continuar el rumbo que Él encomendó para mi vida.

A mi hermana, mi compañera de vida, por ser mi confidente, mi guardespaldas y mi mejor aliada. Gracias por guiarme y apoyarme en cada decisión que he tomado, y por recordarme día a día cuánto valgo y todo lo que soy capaz de construir contando con tu apoyo incondicional.

A mi mamá, por heredarme su más grande virtud: la fortaleza de ser una mujer guerrera.

A mi hermano, cuya visión me inspiró e instruyó en la elección de mi carrera profesional.

A mi papá, quien siempre tuvo la historia perfecta para sacarme una sonrisa y levantarme el ánimo en aquellos días en que el camino se tornaba gris.

A mi gran amigo José Ángel y a la familia Valera López, por abrirme las puertas de su hogar y brindarme un lugar al cual pertenecer. Gracias no solo por apoyarme económicamente cuando más lo necesité, sino por recibirme como a un miembro más de su familia.

A mi gran amiga Ghema, quien me enseñó el verdadero valor de la amistad. Aunque nos acompañamos en los últimos años de la carrera, me demostraste día a día de lo que soy capaz, sosteniendo la fe en mí incluso cuando a mí me costaba creerlo.

Finalmente, a las instalaciones deportivas de la Universidad Simón Bolívar y a todas las personas que se entregan a mantenerlas vivas. Sin la práctica del deporte y sin el voleibol, no habría alcanzado la disciplina y la templanza necesarias para afrontar las exigencias académicas que me permitieron llegar hasta aquí.

RESUMEN

En este trabajo de pasantía se diseñó, implementó y validó un sistema de Internet Industrial de las Cosas (IIoT, por sus siglas en inglés, *Industrial Internet of Things*) de tres niveles (*Edge*, *Gateway* y *Cloud*) para la supervisión remota y diagnóstico determinista en tiempo real de impresoras fiscales en la empresa The Factory HKA. La arquitectura integra un nodo transmisor basado en el microcontrolador ESP32 ejecutado sobre el *Espressif IoT Development Framework* (ESP-IDF) y el sistema operativo de tiempo real *Free Real-Time Operating System* (FreeRTOS), junto a una plataforma web reactiva desarrollada en Django y *HTML eXtensions* (HTMX). La captura local de registros de estado (*Status S1*) se realiza mediante una interfaz síncrona en el borde, garantizando determinismo sin alterar la máquina de estados fiscal. La mensajería bidireccional utiliza el protocolo *Message Queuing Telemetry Transport* (MQTT v5.0) sobre capas de seguridad *Transport Layer Security* (TLS 1.2), reduciendo el consumo de ancho de banda en un 80.2 % frente a arquitecturas *Hypertext Transfer Protocol* (HTTP) tradicionales. Asimismo, mediante optimizaciones de configuración en el microcontrolador, se redujo la ocupación de memoria RAM de instrucciones (IRAM) del 78.39 % al 52.26 %. El sistema incorpora un mecanismo inalámbrico de actualización remota (*Over-The-Air*, OTA) de doble objetivo, capaz de reescribir in situ tanto el firmware del *gateway* como el del periférico fiscal cliente a través de un puente síncrono. Los resultados confirman un monitoreo de alta disponibilidad, resiliencia ante cortes de red y una drástica optimización del mantenimiento preventivo industrial.

Palabras clave: IIoT, ESP32, MQTT v5.0, HTMX, Equipos Fiscales, Firmware Embebido, OTA.

ÍNDICE GENERAL

DEDICATORIA iii

AGRADECIMIENTOS iv

RESUMEN v

ÍNDICE GENERAL vi

ÍNDICE DE FIGURAS ix

ÍNDICE DE TABLAS xi

LISTA DE ACRÓNIMOS xii

NOTACIÓN MATEMÁTICA xiv

INTRODUCCIÓN 1

CAPÍTULO I: MARCO TEÓRICO 4

1.1. Internet de las Cosas (IoT) en el Ámbito Industrial 4
1.2. Ecosistema de Impresión Fiscal en Venezuela 5
1.2.1. Arquitectura modular y Gestión de Memorias 6
1.2.2. Interconexión de Datos: El Rol del Status S1 y SV2 7
1.3. Modelos de Comunicación y Mensajería 7
1.3.1. Arquitectura Publicación-Suscripción (Pub/Sub) 8
1.3.2. Comunicación Bidireccional Full-Duplex (WebSockets) 9
1.3.3. Modelo Maestro-Esclavo en Buses Seriales 9
1.4. Hardware de Alto Rendimiento: ESP32 10
1.4.1. Arquitectura de Procesamiento y Multitarea (FreeRTOS) 10
1.4.2. Jerarquía de Memoria y Optimización Crítica 11
1.4.3. Seguridad por Hardware y Aceleración Criptográfica 11
1.4.4. Interfaces de Comunicación y Capacidad Sensorial 12
1.5. Desarrollo Profesional con ESP-IDF y FreeRTOS 12
1.5.1. Gestión de Tareas y Sincronización (Mutexes y Colas) 12
1.6. Protocolos de Red y Transporte 13
1.6.1. MQTT v5.0: Características Avanzadas y Trazabilidad 14
1.6.2. Seguridad en la Capa de Transporte (TLS/SSL) 14
1.6.3. Comunicación Serial e Interfaz de Datos Local 15

1.7. Estructuración de Datos con JSON 15
1.8. Integración Web y Monitoreo en Tiempo Real 16
1.8.1. Protocolo WebSockets 16
1.8.2. MQTT sobre WebSockets 16
1.9. Arquitectura de Gestión y Orquestación: Framework Django 17
1.9.1. Patrón de Diseño MVT (Model-View-Template) 17
1.9.2. Arquitectura Web Eficiente: HTMX y la comunicación Asíncrona . 18
1.9.3. Gestión Administrativa y Persistencia 18

CAPÍTULO II: DESARROLLO **19**
2.1. Diseño Conceptual de la Arquitectura IIoT 19
2.2. Análisis de Decisiones de Arquitectura y Compromisos de Ingeniería 22
2.2.1. Microcontrolador de Doble Núcleo: ESP32 22
2.2.2. Unidad de Control Fiscal: Impresora HKA80 23
2.2.3. Justificación del Framework Nativo: Determinismo vs. Latencia (ESP-IDF) 24
2.3. Fase de Prototipado y Simulación Inicial 26
2.3.1. Configuración de la Comunicación Asíncrona (UART) 26
2.3.2. Integración de Mensajería Ligera (MQTT sobre TCP) 28
2.4. Diagnóstico e Identificación de Inestabilidades en Entornos Industriales Hostiles 30
2.4.1. Restricciones de Memoria y Gestión de la pila IRAM 30
2.4.2. Interferencia Electromagnética y Basura en el Bus UART 31
2.4.3. Problemas de Concurrencia (Race Conditions) 32
2.5. Diseño de un Kernel Concurrente e Invariante de Red a Prueba de Condiciones de Carrera 33
2.5.1. Implementación de la Interfaz Síncrona de Alto Rendimiento 33
2.5.2. Orquestación Concurrente mediante FreeRTOS 34
2.5.3. Estrategias de Resiliencia de Red y DNS Injection 36
2.6. Seguridad y Trazabilidad del Canal de Telemetría MQTT v5.0 36
2.6.1. Configuración Segura del Cliente MQTT (TLS 1.2) 37
2.6.2. Trazabilidad mediante User Properties y Last Will and Testament . 38
2.7. Arquitectura de Puente Síncrono de Reescritura Remota para Periféricos In Situ (OTA Doble Objetivo) 39
2.7.1. Actualización del Módulo Gateway (ESP32) 40
2.7.2. Actualización del Firmware del Periférico Fiscal a través de la Interfaz Síncrona Local 41
2.7.3. Máquina de Estados Finita para el Puente Síncrono Local 42
2.8. Ecosistema Web de Gestión y Visualización 42
2.8.1. Arquitectura del Backend: Worker MQTT Independiente 43
2.8.2. Persistencia de Datos en SQLite y Lógica de Registro de Logs . . . 44
2.8.3. Interfaz Reactiva SPA mediante Django y HTMX 45

CAPÍTULO III: RESULTADOS **47**
3.1. Implementación del Prototipo Funcional de Borde 47
3.1.1. Entorno de Pruebas Hardware-in-the-Loop (HIL) 48
3.2. Evaluación Cuantitativa de la Comunicación IoT y Telemetría 50
3.2.1. Mecanismo de Extracción Transparente sin Interrupción del Ciclo Fiscal 50
3.2.2. Purga de Caracteres Invisibles y Sanitización del Payload 52
3.2.3. Inmunidad Electromagnética y Robustez del Bus Local 53
3.2.4. Consumo de Ancho de Banda: HTTP vs. MQTT v5.0 55
3.2.5. Resiliencia de Red y Mecanismo Last Will and Testament (LWT) . 55
3.2.6. Validación de Casos Borde y Recuperación ante Micro-cortes 56
3.3. Desempeño del Ecosistema Web y Matriz de Latencias E2E 58
3.3.1. Caracterización de la Latencia End-to-End (T_{E2E}) 58
3.3.2. Monitor en Tiempo Real (Dashboard) y Log Capture 60
3.4. Pruebas del Sistema de Actualización Remota (OTA) 61
3.4.1. Auto-actualización del Módulo Gateway (ESP32) 61
3.4.2. Puente Síncrono Heterogéneo de Reescritura de Firmware In Situ . 62
3.5. Análisis de Rendimiento y Estabilidad Global del Sistema 64
3.5.1. Perfilado Formal de Memoria IRAM y DRAM 65
3.5.2. Perfilado Térmico y Energético del Subclocking (80 MHz) 65
3.5.3. Trazabilidad y Estabilidad de Almacenamiento en Servidor SQLite 68
3.6. Cumplimiento de los Objetivos Específicos 69

CONCLUSIONES **71**

REFERENCIAS **73**

APÉNDICE A: CÓDIGO FUENTE DEL SISTEMA **75**
A.1. Firmware del Módulo de Transmisión IoT (MT-IoT) 75
A.1.1. Sanitización y Parseo de la Trama SV2 75
A.1.2. Interrogación Serial No Bloqueante 76
A.1.3. Validación de Integridad de Trama 76
A.1.4. Exclusión Mutua de Memoria y Bus Físico 77
A.1.5. Resiliencia y Reconexión MQTT 78
A.1.6. Configuración Segura del Cliente MQTT 78
A.1.7. Inyección de Metadatos mediante User Properties 78
A.1.8. Testamento (Last Will and Testament) 79
A.1.9. Reanudación de Descargas mediante HTTP Range 79
A.1.10. Empaquetado de Firmware Segmentado hacia el Periférico 79
A.2. Estructuras de Mensajería (JSON) 80
A.2.1. Telemetría del Status S1 80
A.2.2. Comando de Actualización Remota 81
A.3. Herramientas de Simulación 81
A.4. Worker de Telemetría en el Servidor 81
A.5. Interfaz Reactiva Web 82

ÍNDICE DE FIGURAS

1.1. Diagrama de los tres niveles fundamentales en un sistema IIoT. 5
1.2. Estructura de la Impresora Fiscal. 6
1.3. Modelo MQTT de publicación y suscripción. 8
1.4. Vista realista del microcontrolador ESP32. 10
1.5. Diagrama de ejecución de tres tareas en distintos instantes de tiempo. . . . 13

2.1. Flujo metodológico general del trabajo de pasantía organizado en cuatro fases sucesivas. 19
2.2. Diagrama de la Arquitectura IIoT distribuida en tres niveles (Edge, Gateway y Cloud). 21
2.3. Diagrama propuesto para la arquitectura de comunicaciones MQTT. 21
2.4. Pinlayout de la tarjeta ESP32 DevKitC V4 y reasignación de pines seriales. 22
2.5. Configuración aplicada en menuconfig para optimización de IRAM. 23
2.6. Ajuste de frecuencia de reloj del CPU a 80 MHz en `menuconfig` para estabilidad térmica en operación continua. 23
2.7. Impresora fiscal modelo HKA80 autorizada por la normativa vigente. . . . 24
2.8. Inclusión estricta de dependencias en el archivo de construcción CMakeLists.txt. 26
2.9. Captura de inyección manual de tramas y comando ENQ mediante la terminal de datos RealTerm. 27
2.10. Máquina de estados del simulador fiscal en software. 28
2.11. Perfilado de memoria revelando un nivel crítico de ocupación de la IRAM (78.39 %). 31
2.12. Traza de depuración y registro de evento ante colisión por acceso concurrente al bus local (Choque de Trenes). 33
2.13. Interacción maestro-esclavo sobre el bus síncrono local, con señal de reloj dedicada a 100 kHz y vías independientes de transmisión y recepción. . . . 34
2.14. Exclusión mutua en dos niveles para la protección de la memoria RAM y del bus físico. 35
2.15. Estructura de empaquetado de un fragmento de firmware hacia el periférico local. 42
2.16. Máquina de estados del bridge síncrono para reprogramación de periféricos locales. 43
2.17. Flujo de actualización reactiva del DOM mediante eventos MQTT y atributos HTMX. 45

3.1. Prototipo funcional del MT-IoT integrado a la impresora fiscal en entorno de pruebas. 48
3.2. Arquitectura IIoT definitiva del sistema, desplegada en borde y nube. . . . 49
3.3. Montaje del banco de pruebas Hardware-in-the-Loop: el ESP32 maestro (MT-IoT) y el ESP32 esclavo que emula al periférico fiscal, interconectados sobre el bus síncrono local. 50
3.4. Esquema definitivo de rutas y mensajería MQTT v5.0 del sistema. 51
3.5. Traza de depuración en consola serial evidenciando el desempaquetado de la trama S1. 52
3.6. Registro en la vista `Log Capture` confirmando la intercepción de una desconexión LWT. 56
3.7. Diagrama de flujo de la resiliencia de red y del ciclo de detección, reconexión y restauración del túnel MQTT. 57
3.8. Panel de Control General (Dashboard) operando bajo arquitectura SPA reactiva. 61
3.9. Visualización en `Log Capture` de un cambio de estado de error mecánico (`0x48`). 62
3.10. Diagrama de flujo de la orquestación de la actualización remota (OTA) de doble objetivo. 63
3.11. Intermediación web desplegando una actualización de firmware crítica hacia el MT-IoT. 64
3.12. Confirmación asíncrona en la interfaz web de actualización exitosa en la impresora fiscal. 65
3.13. Perfilado de memoria inicial revelando ocupación crítica de IRAM (78.39 %). 66
3.14. Ocupación de IRAM optimizada al 52.26 % tras deshabilitar aceleración Wi-Fi. 67

ÍNDICE DE TABLAS

2.1. Análisis comparativo de entornos de desarrollo para el nodo Gateway. . . . 25
2.2. Estructura del mensaje JSON del `Status S1` publicado hacia el *broker* MQTT. . . . 29
2.3. Parámetros de configuración del cliente MQTT con seguridad TLS estricta. 38
2.4. Propiedades de usuario (*User Properties*) inyectadas en cada publicación MQTT v5.0. . . . 39
2.5. Estructura del mensaje JSON de comando para iniciar una actualización remota. . . . 40

3.1. Efecto de la purga de caracteres invisibles sobre el campo de versión del periférico. . . . 53
3.2. Niveles de defensa implementados para la inmunidad electromagnética del bus local. . . . 54
3.3. Comparativa cuantitativa del consumo de ancho de banda por trama de telemetría fiscal. . . . 55
3.4. Validación de casos borde y recuperación de estado ante fallas de energía y red. . . . 58
3.5. Matriz de latencias, propagación de incertidumbre y tiempos de respuesta E2E del ciclo de telemetría. . . . 59
3.6. Perfilado de recursos de memoria IRAM y DRAM en el SoC ESP32 ante optimizaciones de `menuconfig`. . . . 66
3.7. Compromiso térmico-energético del subclocking del SoC ESP32. . . . 68
3.8. Métricas de trazabilidad y desempeño del almacenamiento en SQLite. . . . 69
3.9. Matriz de cumplimiento de objetivos específicos y alcance técnico. . . . 70

LISTA DE ACRÓNIMOS

ACK	*Acknowledgment* (Acuse de Recibo)
AES	*Advanced Encryption Standard* (Estándar de Cifrado Avanzado)
AJAX	*Asynchronous JavaScript and XML* (JavaScript y XML Asíncronos)
API	*Application Programming Interface* (Interfaz de Programación de Aplicaciones)
CRUD	*Create, Read, Update, Delete* (Crear, Leer, Actualizar y Borrar)
DEP	Decanato de Estudios Profesionales (Universidad Simón Bolívar)
DMA	*Direct Memory Access* (Acceso Directo a Memoria)
DNS	*Domain Name System* (Sistema de Nombres de Dominio)
DOM	*Document Object Model* (Modelo de Objetos del Documento)
DRAM	*Data Random Access Memory* (Memoria de Acceso Aleatorio para Datos)
ENQ	*Enquiry* (Carácter de Consulta o Interrogación)
ESP-IDF	*Espressif IoT Development Framework* (Marco de Desarrollo IoT de Espressif)
ETX	*End of Text* (Carácter de Fin de Texto de Trama Serial)
FreeRTOS	*Free Real-Time Operating System* (Sistema Operativo de Tiempo Real Libre)
GPIO	*General Purpose Input/Output* (Entrada/Salida de Propósito General)
HIL	*Hardware-in-the-Loop* (Hardware en el Bucle de Simulación)
HTMX	*HTML eXtensions* (Extensiones para HTML)
HTTP	*Hypertext Transfer Protocol* (Protocolo de Transferencia de Hipertexto)
IIoT	*Industrial Internet of Things* (Internet Industrial de las Cosas)
IoT	*Internet of Things* (Internet de las Cosas)
IP	*Internet Protocol* (Protocolo de Internet)
IRAM	*Instruction Random Access Memory* (Memoria de Acceso Aleatorio para Instrucciones)
JSON	*JavaScript Object Notation* (Notación de Objetos de JavaScript)
LRC	*Longitudinal Redundancy Check* (Verificación de Redundancia Longitudinal)
LWT	*Last Will and Testament* (Mecanismo de Última Voluntad y Testamento)
MQTT	*Message Queuing Telemetry Transport* (Transporte de Telemetría por Encolamiento de Mensajes)
MT-IoT	Módulo de Transmisión IoT (Nodo de transmisión basado en ESP32)
MVT	*Model-View-Template* (Patrón de Arquitectura Modelo-Vista-Plantilla)
NAK	*Negative Acknowledgment* (Acuse de Recibo Negativo)
NVS	*Non-Volatile Storage* (Almacenamiento No Volátil)
ORM	*Object-Relational Mapping* (Mapeo Objeto-Relacional)
OTA	*Over-The-Air* (Actualización Inalámbrica de Firmware)
PDF	*Portable Document Format* (Documento en Formato Portable)

PEM	*Privacy Enhanced Mail* (Formato de Certificados con Privacidad Mejorada)
QoS	*Quality of Service* (Calidad de Servicio)
RIF	Registro Único de Información Fiscal
RS232	*Recommended Standard 232* (Estándar Recomendado 232, Interfaz Serial)
RSSI	*Received Signal Strength Indicator* (Indicador de Intensidad de Señal Recibida)
SENIAT	Servicio Nacional Integrado de Administración Aduanera y Tributaria
SHA-2	*Secure Hash Algorithm 2* (Algoritmo de Hash Seguro 2)
SoC	*System on Chip* (Sistema en Chip)
SPA	*Single Page Application* (Aplicación de Página Única)
SPI	*Serial Peripheral Interface* (Interfaz Serial Periférica)
SQL	*Structured Query Language* (Lenguaje de Consulta Estructurado)
SSL	*Secure Sockets Layer* (Capa de Conexión Segura)
STX	*Start of Text* (Carácter de Inicio de Texto de Trama Serial)
TCP	*Transmission Control Protocol* (Protocolo de Control de Transmisión)
TLS	*Transport Layer Security* (Seguridad en la Capa de Transporte)
UART	*Universal Asynchronous Receiver-Transmitter* (Transmisor-Receptor Asíncrono Universal)
USB	Universidad Simón Bolívar / *Universal Serial Bus* (Bus Serial Universal)
WSS	*WebSocket Secure* (WebSocket Seguro)

NOTACIÓN MATEMÁTICA

Símbolos Latinos

B	Contador de bytes recibidos durante la descarga [bytes]
b_i	i-ésimo byte de la trama fiscal [byte]
C_{ef}	Capacitancia efectiva conmutada del circuito [F]
f	Frecuencia de operación del núcleo [MHz]
i_0	Índice inicial del campo de versión en el búfer [−]
L_{fw}	Longitud del campo de versión del *firmware* [caracteres]
n	Longitud total de la trama fiscal [bytes]
$O(1)$	Complejidad temporal constante [−]
P	Potencia total disipada por el SoC [W]
P_{din}	Potencia dinámica disipada [W]
p_{ETX}	Posición del carácter terminador ETX en el búfer [índice]
$R_{\theta j-a}$	Resistencia térmica unión-ambiente [°C/W]
T_{E2E}	Latencia extremo a extremo del ciclo de telemetría [ms]
T_{JSON_pack}	Tiempo de empaquetado y serialización del *payload* [ms]
T_{MQTT_pub}	Tiempo de publicación y tránsito por el *broker* [ms]
$T_{TLS_handshake}$	Tiempo de establecimiento seguro del canal TLS [ms]
T_{UART_poll}	Tiempo de sondeo del registro *Status S1* en el bus local [ms]
T_{WS_render}	Tiempo de renderizado reactivo del DOM [ms]
T_r	Retardo de cortesía antes de reintentar la reconexión [ms]
T_s	Intervalo de sondeo fiscal [ms]
V_{dd}	Tensión de alimentación del núcleo [V]

Símbolos Griegos

ΔT_{j-a}	Salto térmico entre la unión y el ambiente [°C]
η_{banda}	Eficiencia de reducción del ancho de banda [%]
σ	Desviación estándar de una medición [ms]
σ_{E2E}	Desviación estándar de la latencia extremo a extremo [ms]
σ_i	Desviación estándar de la i-ésima etapa [ms]

Subíndices y Superíndices

banda	Magnitud referida al ancho de banda
calc	Valor calculado por el *firmware*
dd	Alimentación del núcleo (*drain-to-drain*)
din	Naturaleza dinámica de la magnitud
E2E	Extremo a extremo (*end-to-end*)
ef	Valor efectivo
ETX	Carácter de fin de trama (*End of Text*)
fw	Referido al *firmware*
i	Índice de etapa o de byte
j-a	Unión-ambiente (*junction-to-ambient*)
JSON_pack	Empaquetado del mensaje JSON
MQTT_pub	Publicación MQTT
r	Reconexión
rx	Valor recibido
s	Sondeo (*polling*)
TLS_handshake	Establecimiento del canal TLS
UART_poll	Sondeo del bus local
WS_render	Renderizado del DOM

INTRODUCCIÓN

Antecedentes

Históricamente, la fiscalización en Venezuela ha dependido de equipos físicos estáticos (impresoras fiscales) que operan bajo protocolos de comunicación punto a punto, predominantemente el estándar RS232 (UART). Si bien estos equipos han evolucionado en capacidad de almacenamiento de memorias fiscales, su arquitectura de comunicación se ha mantenido aislada de las redes corporativas. Tradicionalmente, la única forma de interactuar con el equipo para diagnóstico o auditoría ha sido la conexión física de un técnico mediante herramientas de depuración serial locales. Con la masificación del Internet de las Cosas (IoT) y el uso de microcontroladores de alto rendimiento como el ESP32, surge la posibilidad de transformar estos periféricos "mudos" en nodos inteligentes. La empresa The Factory HKA, líder en el sector, ha iniciado una transición hacia la digitalización de estos procesos, buscando centralizar la información que antes residía únicamente en el hardware físico de cada contribuyente.

Planteamiento del problema

El laboratorio de desarrollo y la unidad de soporte de The Factory HKA enfrentan limitaciones operativas debido a que el monitoreo de los estados de la impresora (como el Status S1) depende de la presencia física o de conexiones seriales punto a punto. Esta arquitectura dificulta la detección temprana de errores críticos, tales como "fin de papel" "memoria fiscal llena" o fallos mecánicos, que pueden interrumpir la actividad comercial de forma imprevista. Además, la visualización actual de los datos fiscales es fragmentada; los técnicos solo pueden observar el estado de un equipo a la vez mediante herramientas de depuración serial, lo que impide tener una visión global y simultánea del parque de máquinas instaladas.

Justificación

El diseño de un dispositivo basado en el microcontrolador ESP32 funciona como un instrumento de medición digital que moderniza la interacción con los equipos fiscales. Esta solución es beneficiosa en tres aspectos fundamentales: en primer lugar, ofrece una usabilidad superior al eliminar la dependencia de cables seriales hacia una PC local; en segundo lugar, permite una visualización centralizada mediante un dashboard web que muestra múltiples equipos en simultáneo, facilitando el análisis de patrones; y finalmente, representa una evolución digital para la empresa, integrando protocolos industriales modernos como MQTT v5.0. La implementación de una base de datos de series de tiempo para almacenar los históricos de ventas y errores permite, además, realizar mantenimiento preventivo basado en datos reales.

Objetivo general

Diseñar, implementar y validar un sistema basado en microcontroladores (ESP32) que permita el monitoreo remoto y en tiempo real de los datos operativos de equipos fiscales, garantizando la confiabilidad y robustez del sistema de comunicación.

Objetivos específicos

- Diseñar la arquitectura del sistema IoT que integre los microcontroladores, los protocolos de comunicación y la plataforma de monitoreo remoto.
- Desarrollar el firmware embebido para los microcontroladores ESP32.
- Integrar el sistema con sensores o interfaces de los equipos fiscales, garantizando la captura de datos relevantes.
- Implementar la comunicación mediante MQTT con un servidor externo.
- Implementar un servidor MQTT para recibir y distribuir los datos enviados por los dispositivos.
- Desarrollar una interfaz web o móvil (dashboard) que permita visualizar en tiempo real el estado de los equipos fiscales.

- Validar el sistema, evaluando la confiabilidad de la comunicación, la latencia, la frecuencia de actualización y la robustez ante fallos de red o energía.
- Documentar el proceso de desarrollo, incluyendo diagramas de arquitectura, flujos de datos, scripts de prueba y resultados de validación.

CAPÍTULO 1

MARCO TEÓRICO

1.1. Internet de las Cosas (IoT) en el Ámbito Industrial

El **Internet Industrial de las Cosas (IIoT)** representa la convergencia de las tecnologías de la información (IT) y las tecnologías operativas (OT), posicionándose como el motor fundamental de la **Cuarta Revolución Industrial** [17]. A diferencia del IoT orientado al consumidor, el IIoT se define como la aplicación de estas tecnologías en entornos industriales o empresariales para crear valor a través de procesos, cadenas de suministro, productos y servicios inteligentes [2]. Esta evolución trasciende la interconexión de objetos cotidianos para enfocarse en la red de equipos, máquinas, computadoras y personas, permitiendo operaciones industriales inteligentes mediante el uso de análisis de datos avanzados.

La complejidad inherente del IIoT radica en que sus soluciones no operan de forma aislada, sino que se integran en sistemas operativos de gran escala, creando interdependencias críticas entre sus componentes. Para comprender su funcionamiento, la literatura científica suele estructurar el ecosistema en tres niveles fundamentales (ver figura 1.1) [13]. En el ámbito nacional, antecedentes recientes han demostrado la viabilidad de arquitecturas de monitoreo en tiempo real en el contexto universitario [16].

1. **Funcionalidad de borde (Edge):** Sensores y actuadores conectados físicamente a equipos y materiales.
2. **Pasarelas de datos (Gateways):** Mecanismos encargados de la recepción y transmisión transparente de información entre los dispositivos de borde y los servidores en red.
3. **Gestión y análisis de datos:** Sistemas basados habitualmente en la nube para la recopilación, enlace y análisis de la información para la toma de decisiones.

Un pilar en esta arquitectura es la transformación de equipos convencionles en **productos inteligentes**. Según la Universidad de Cambridge, un activo físico integrado en

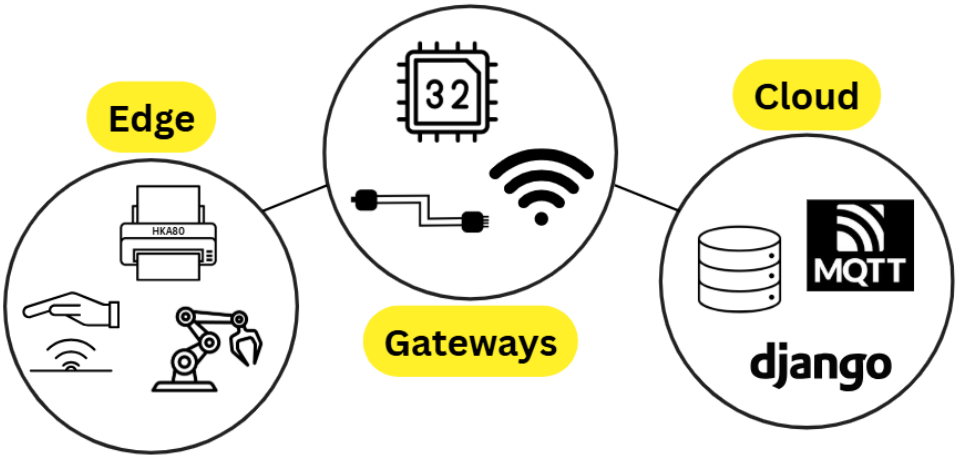


Figura 1.1. Diagrama de los tres niveles fundamentales en un sistema IIoT.

IIoT posee identidad única [2], capacidad de comunicación efectiva con su entorno, almacenamiento de datos propio y autonomía para participar en decisiones relevantes para su funcionamiento. Esta metamorfosis de sistemas previamente aislados hacia una red conectada permite recolectar datos "no productivos" (como diagnóticos de mantenimiento o estados operativos) que antes eran inaccesibles, optimizando así el ciclo de vida del producto y la eficiencia general del ecosistema industrial.

Finalmente, la transición de equipos aislados a sistemas hiperconectados introduce una superficie de ataque significativamente mayor, donde la seguridad deja de ser una característica opcional para convertirse en un requisito de diseño mandatorio. El World Economic Forum advierte que el estado de vulnerabilidad en el sector industrial es insostenible sin la adopción de protocolos de seguridad y protección activa [17]. En el contexto fiscal, donde la integridad de los datos es innegociable, el ecosistema IIoT requiere mecanismos de "endurecimiento activo" (active hardening) que garanticen la tríada de la seguridad: confidencialidad, integridad y disponibilidad.

1.2. Ecosistema de Impresión Fiscal en Venezuela

En el marco legal venezolano, una **impresora fiscal** no es simplemente un periférico de salida, sino una unidad de control autorizada por el **SENIAT** para garantizar la integridad de la recaudación tributaria [14]. Bajo normativas como la Providencia 0141, estos equipos son los responsables de emitir documentos con validez legal (facturas, notas de crédito

y reportes de cierre diario o "Reporte Z"). El corazón de su funcionamiento reside en el *firmware*, un software embebido que regula la interacción entre el hardware de impresión y los módulos de memoria protegida, asegurando que ninguna transacción pueda ser alterada una vez registrada [14, 15].

1.2.1. Arquitectura modular y Gestión de Memorias

La arquitectura de una impresora fiscal, como la **HKA80** utilizada en este proyecto, se divide en dos bloques críticos que garantizan la trazabilidad total de las operaciones (vease figura 1.2) [14]:

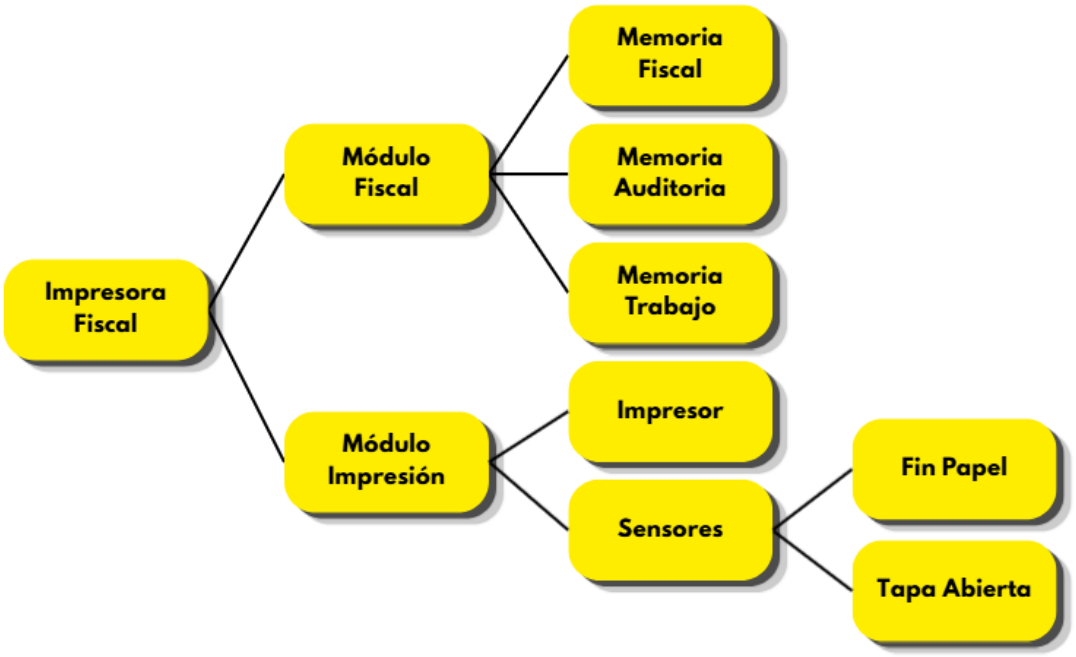


Figura 1.2. Estructura de la Impresora Fiscal.

1. **Módulo de Impresión:** Compuesto por el cabezal térmico y sensores de estado (fin de papel, tapa abierta).

2. **Módulo fiscal:** El componente más sensible del equipo, encargado de la gestión de tres tipos de memorias independientes:

   - **Memoria Fiscal (MF):** Dispositivo inalterable adherido al chasis que almacena los resúmenes de los Reportes Z, con una cantidad típica de 2000 cierres diarios.

- **Memoria de Auditoría (MA):** Unidad de almacenamiento masivo (generalmente de 2 GB) que guarda copia electrónica fidedigna de cada documento impreso [14].
- **Memoria de Trabajo:** Memoria volátil que mantiene los acumuladores de ventas y contadores de la jornada actual, los cuales se transfieren a la MF al ejecutar el cierre diario [14].

1.2.2. Interconexión de Datos: El Rol del Status S1 y SV2

El monitoreo remoto planteado en este trabajo requiere una interconexión física de alto rendimiento para captura de datos en tiempo real. Si bien los sistemas administrativos tradicionales utilizan el estándar serial RS232 para la gestión local [14], para la integración directa entre el microcontrolador ESP32 y el hardware de la impresora se implementó una **interfaz de comunicación síncrona** del tipo SPI.

Esta arquitectura de comunicación permite al ESP32 actuar como el controlador principal de la línea, gestionando el flujo de datos mediante señales de sincronismo y control dedicadas. A través de este bus, el firmware realiza el *polling* de información fiscal solicitando tramas específicas como el **Status S1** (Serial del equipo, RIF y contadores) y **SV2** (versión de firmware de la impresora) [14, 15]. La migración no solo garantiza una comunicación de baja latencia y alta confiabilidad, sino que asegura la escalabilidad del sistema ante futuras actualizaciones de hardware, manteniendo la compatibilidad con el dispositivo de transmisión actual de la empresa.

1.3. Modelos de Comunicación y Mensajería

En el desarrollo de ecosistemas IIoT, la eficiencia del intercambio de datos depende de la selección de modelos que optimicen el uso del ancho de banda y garantice la integridad de la información. El sistema de transmisión integra tres arquitecturas fundamentales que operan en diferentes niveles de la pila de comunicación, desde la interacción física con el hardware fiscal hasta la visualización en tiempo real en la nube [12].

1.3.1. Arquitectura Publicación-Suscripción (Pub/Sub)

A diferencia del modelo tradicional de consulta (request-response), el patrón **Publicación-Suscripción** desacopla a los productores de datos de los consumidores mediante un intermediario denominado *Broker*, como se muestra en la figura 1.3 [12]. En este proyecto, el microcontrolador ESP32 actúa como publicador de telemetría fiscal, mientras que el Dashboard web se suscribe a tópicos específicos (ej. `v1/fiscal/serial/json`) para recibir información.

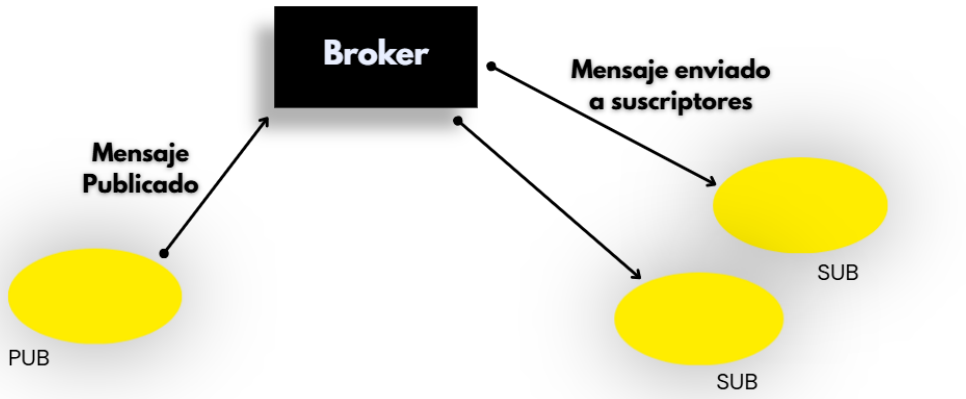


Figura 1.3. Modelo MQTT de publicación y suscripción.

Este modelo ofrece tres dimensiones de desacoplamiento críticas para la robustez industrial [12]:

- **Espacial:** Los dispositivos no necesitan conocer la dirección IP de los demás para comunicarse.
- **Temporal:** No es necesario que el suscriptor esté en línea al momento de la publicación si se utilizan sesiones persistentes.
- **Sincronización:** Las operaciones en el ESP32 no se bloquean mientras se envían los datos, permitiendo que la tarea de la impresora continúe su ejecución.

1.3.2. Comunicación Bidireccional Full-Duplex (WebSockets)

Para el monitoreo en tiempo real, se implementó el protocolo **WebSockets**, el cual establece un canal de comunicación persistente y bidireccional sobre una única conexión TCP [8, 10]. A diferencia de HTTP, que requiere una nueva cabecera por cada solicitud, WebSockets reduce drásticamente la latencia mediante un proceso de *handshake* que eleva la conexión de HTTP a un flujo de datos binarios continuo.

En la arquitectura final de este proyecto se implementa la técnica de túnel o encapsulamiento de **MQTT sobre WebSockets** [12, 10]. Esta integración es fundamental debido a que los navegadores web no permiten la apertura de conexiones TCP crudas (raw sockets), las cuales son requeridas por el protocolo MQTT nativo [8]. Al encapsular los paquetes MQTT dentro de tramas de WebSocket, el Dashboard web puede suscribirse e interactuar directamente con el broker (HiveMQ Cloud) de forma bidireccional y cifrada (WSS) [12]. Esto elimina la necesidad de implementar servidores intermediarios complejos (como Django Channels), simplificando la infraestructura backend y garantizando la recepción en tiempo real de los estados y alertas del parque de impresoras.

1.3.3. Modelo Maestro-Esclavo en Buses Seriales

En el nivel del borde (*Edge*), la interacción entre el microcontrolador y la impresora fiscal se rige por el modelo **Maestro-Esclavo** [14]. En esta jerarquía, el ESP32 asume el rol de maestro, teniendo el control total sobre el bus de comunicación y decidiendo cuando iniciar la transferencia de datos.

Para la integración con la impresora HKA80, este modelo se manifiesta a través de un **bus de comunicación serie síncrono** (operando bajo una arquitectura Maestro-Esclavo a 100kHz). El controlador principal (ESP32) genera la señal de temporización (*Clock*) y gestiona la habilitación del dispositivo periférico para realizar un sondeo (*polling*) constante de los registros fiscales. Si bien este modelo presenta una alta dependencia del maestro, garantiza un determinismo absoluto en la captura de datos críticos al eliminar las desviaciones por deriva de reloj (*baud-rate drift*) y la inyección de bytes corruptos por ruido electromagnético que afectaban a la interfaz asíncrona tradicional. Esto asegura que cada trama de **Status S1** sea recibida y validada mediante comprobación de redundancia (LRC) antes de proceder con la lógica de red.

1.4. Hardware de Alto Rendimiento: ESP32

El núcleo del sistema de transmisión se fundamenta en la tarjeta de desarrollo **ESP32 DevKitC V4** (basada en el módulo **ESP32-WROOM-32**) [7]. Este dispositivo integra un sistema en chip (SoC) de bajo consumo diseñado por Espressif Systems que incorpora conectividad Wi-Fi e interfaz Bluetooth de modo dual. A diferencia de los microcontroladores convencionales, este componente está concebido para aplicaciones industriales de IoT, ofreciendo una robustez y versatilidad que permiten manejar simultáneamente tareas de comunicación segura y el control de periféricos de alta velocidad (véase la Figura 1.4).



Figura 1.4. Vista realista del microcontrolador ESP32.

1.4.1. Arquitectura de Procesamiento y Multitarea (FreeRTOS)

En la variante específica utilizada para este desarrollo, el dispositivo integra un microprocesador de arquitectura **Xtensa® Dual-Core LX6 de 32 bits**, capaz de alcanzar una frecuencia de reloj de hasta 240 MHz y un rendimiento de 600 MIPS [7]. Si bien las variantes más recientes de la familia ESP32 (como las series basadas en arquitectura RISC-V) operan bajo un solo núcleo, la arquitectura de doble núcleo Xtensa LX6 del modelo seleccionado es un pilar crítico en este diseño. Esta arquitectura permite aprovechar las capacidades SMP (Symmetric Multiprocessing) de FreeRTOS para aislar físicamente los procesos de captura de datos locales en un núcleo dedicado, evitando que retardos por

procesamiento del stack de red afecte el tiempo real del sistema.

1.4.2. Jerarquía de Memoria y Optimización Crítica

La gestión de la memoria es el factor determinante para la estabilidad de sistemas que emplean MQTT v5.0 y cifrado TLS/SSL. El ESP32 cuenta físicamente con 520 KB de SRAM interna, la cual se distribuye de forma lógica para optimizar el rendimiento del sistema [7]:

- **IRAM (Instruction RAM):** Espacio de 128 KB reservado para instrucciones críticas y manejadores de interrupciones. El uso eficiente de la IRAM es determinante en sistemas embebidos industriales para alojar stacks de seguridad como Mbed TLS sin comprometer la estabilidad del sistema [7].
- **DRAM (Data RAM):** Utilizada para buffers de datos y el *heap* de la aplicación. Para garantizar la robustez ante fallos de energía, el sistema emplea la **NVS Flash** (memoria no volátil) para almacenar configuraciones críticas que deben persistir tras un reinicio [5].

1.4.3. Seguridad por Hardware y Aceleración Criptográfica

El despliegue de dispositivos IoT en infrestructuras fiscales introduce vulnerabilidades críticas que deben ser mitigadas mediante estándares de crifrado robustos. Sin embargo, la ejecución de algoritmos como AES (Advanced Encryption Standard) o SHA-2 (Secure Hash Algorithm) mediante software consume ciclos de reloj significativos, lo que puede elevar el consumo energético y generar latencia [7].

El uso de **Aceleradores Criptográficos por Hardware** en microcontroladores modernos permite delegar estas operaciones matemáticas complejas a módulos de silicio dedicados [7, 11]. Esto garantiza:

- **Integridad de Datos:** Asegura que los registros fiscales no sean alterados durante el tránsito hacia la nube.
- **Autenticación Mutua:** Facilita el intercambio de certificados digitales en el protocolo TLS/SSL de forma eficiente, permitiendo que solo dispositivos autorizados se comuniquen con el servidor central [3].

1.4.4. Interfaces de Comunicación y Capacidad Sensorial

El sistema de transmisión aprovecha la matriz de periféricos avanzados del chip para la integración con el ecosistema fiscal [7]. Se utiliza una interfaz de comunicación física serie síncrona operando a una **frecuencia de reloj de** 100 **kHz**, lo que asegura un intercanbio de datos constante y de baja latencia con hardware de impresión. Esta interfaz permite al ESP32 actuar como el maestro de la comunicación, gestionando líneas dedicadas para la captura inmediata de registros de estado.

Asimismo, la unidad de radio integrada de 2.4 GHz permite la interconexión bajo los estándares 802.11 b/g/n, posicionando al dispositivo como un node Gateway embebido que cumple con las capas de detección e intercambio de datos descritas en la arquitectura IoT [7].

1.5. Desarrollo Profesional con ESP-IDF y FreeRTOS

A diferencia de los entornos de prototipado rápido, la implementación de un sistema de monitoreo fiscal industrial exige un control granular sobre los recursos del hardware y una ejecución determinista de las tareas. Por ello, este proyecto se fundamenta en el **Espressif IoT Development Framework (ESP-IDF)**, el entorno de desarrollo nativo que permite explotar la arquitectura de doble núcleo del ESP32 mediante el uso de un **Sistema Operativo de Tiempo Real (FreeRTOS)** [5, 1].

1.5.1. Gestión de Tareas y Sincronización (Mutexes y Colas)

En entornos industriales, el monitoreo de equipos fiscales exige una ejecución determinística. El uso de un RTOS (Real-Time Operating System) como FreeRTOS permite la abstracción del hardware mediante la implementación de un planificador de tareas (*Scheduler*) basado en prioridades [1]. Esta arquitectura es fundamental para garantizar que los procesos de comunicación no bloqueen la lectura crítica de datos fiscales.

La robustez del firmware se fundamenta en tres pilares conceptuales de la computación concurrente haciendo que las tareas se fragmenten tal como muestra la figura 1.5:

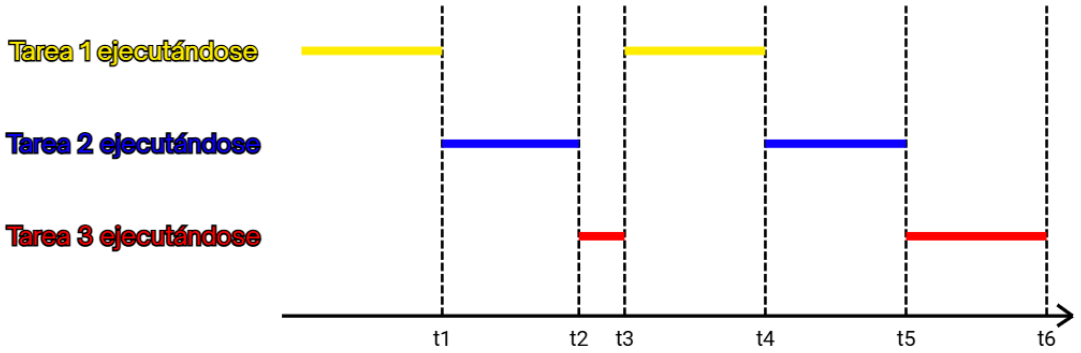


Figura 1.5. Diagrama de ejecución de tres tareas en distintos instantes de tiempo.

- **Planificación por Prioridades (Preemptive Scheduling):** Permite que el procesador alterne entre múltiples hilos de ejecución. En sistema IoT, esto asegura que las tareas de alta prioridad, como el procesamiento de tramas de comunicación como la impresora, se ejecuten imediantamente, mientras que las tareas de menor prioridad, como el envío de telemetría, se procesen en los tiempos de espera del CPU [1].
- **Mecanismos de Exclusión Mutua (Mutex):** El acceso concurrente a recursos compartidos, como los buses de comunicación, representa un riesgo de corrupción de datos. El uso de *Mutexes* garantiza que solo una tarea posea el control del recurso en un momento dado, evitando condiciones de carrera (*race conditions*) que podrían comprometer la integridad de la información tributaria [1].
- **Sincronización mediante Semáforos y Colas (Queues):** Estos mecanismos permiten la comunicación inter-tareas. Las colas de mensajes facilitan el paso de datos entre el núcleo que gestiona la lógica de la impresora y el núcleo encargado de la pila de red, permitiendo una arquitectura desacoplada y escalable [1].

1.6. Protocolos de Red y Transporte

La arquitectura de comunicaciones del sistema de transmisión se fundamenta en una pila de protocolos diseñados para entornos de alta confiabilidad y recursos restringidos. La integración del estándar **MQTT v5.0** sobre capas de seguridad **TLS/SSL**, sumada a la eficiencia del bus para la comunicación local, constituye una infraestructura robusta que garantiza la integridad de la información fiscal desde el borde hasta la nube [12, 3].

1.6.1. MQTT v5.0: Características Avanzadas y Trazabilidad

El protocolo **MQTT (Message Queuning Telemetry Transport)** es un estándar de mensajería basado en el modelo publicación-suscripción, optimizado para redes de baja tasa de bits y alta latencia [12]. Para este proyecto, se implementó la versión **5.0**, la cual representa una evolución crítica respecto a su predecesor (v3.1.1), introduciendo características de escalabilidad y reporte de errores superiores.

Una de las herramientas integradas en el firmware es el uso de las **User Properties**. Estas permiten adjuntar metadatos personalizados en formato clave-valor a las tramas de publicación (PUBLISH), facilitando una **trazabilidad transparente** en sistemas complejos. En el sistema de transmisión, estas propiedades se utilizan para enviar información de autodiagnóstico y modelo de hardware sin sobrecargar el *payload* JSON del mensaje. Además, el uso de **Reason Codes** en todos los paquetes de reconocimiento (ACKs) permite que el ESP32 reciba una retoralimentación detallada del broker ante cualquier fallo de comunicación, mejorando la resilencia operativa del nodo sensor [12].

1.6.2. Seguridad en la Capa de Transporte (TLS/SSL)

Dado que el sistema maneja información tributaria sensible, la seguridad es un requisito de diseño mandatorio. El sistema utiliza el protocolo **Transport Layer Security (TLS)**, su sucesor del Secure Socket Layer (SSL), para proporcionar un canal cifrado sobre TCP que garantiza la **confidencialidad, integridad, y autenticidad** de los datos [3, 11].

La implementación se basa en un proceso de **Handshake** mediante el cual el ESP32 y el broker negocian parámetros criptográficos y validan identidades a través de certificados digitales. Siguiendo las recomendaciones internacionales, el sistema opera sobre el **puerto seguro 8883** (IANA: *secure-mqtt*), lo que mitiga riesgos criticos como el espionaje pasivo (*eavesdropping*) y la inyección de contenido malicioso [11]. Esta capa de seguridad asegura que, incluso si las tramas son interceptadas en la red pública, la información permanezca ininteligible para atacantes externos.

1.6.3. Comunicación Serial e Interfaz de Datos Local

En el nivel de borde, la captura de datos fiscales se realiza a través una **interfaz de comunicación síncrona de alto rendimiento** [14]. A diferencia de las interfaces seriales asíncronas tradicionales (UART), este método permite el intercambio de información en modo *full-duplex* con un determinismo superior.

El sistema de transmisión opera bajo una jerarquía de **Maestro-Esclavo**, donde el microcontrolador ESP32 actúa como el nodo maestro que genera la señal de **sincronización (o temporización)** y gestiona la línea de **habilitación del periférico** para orquestar la comunicación con la impresora fiscal. Esta arquitectura garantiza un determinismo absoluto en la captura de registros críticos (Status S1/SV2), ya que el uso de **vías de transmisión y recepción independientes** evita colisiones de datos y permiten velocidades de transferencia significativamente superiores a las de una conexión RS232 convenicional [14]. El uso de este bus síncrono es lo que permite que el sistema mantenga una frecuencia de muestreo constante de 5 segundos sin interferir con las operaciones internas del firmware de la impresora.

1.7. Estructuración de Datos con JSON

En la arquitectura de comunicaciones de este proyecto, la representacón y el empaquetado de la información fiscal se fundamentan en el formato **JSON (JavaScript Object Notation)**. Este es un estándar de intercambio de datos ligero, basado en una estructura de pares clave-valor, que permite una legibilidad superior tanto para humanos como para máquinas [13]. Su adopción en sistemas IIoT es crítica debido a su independencia de lenguaje, lo que facilita la interoperabilidad entre el firmware desarrollado en **C (ESP-IDF)**, el broker MQTT y el backend en **Python (Django)**.

A diferencia de otros formatos de intercambio de datos basados en texto como XML, JSON (*JavaScript Object Notation*) ofrece una sobrecarga (*overhead*) de sintaxis significativamente menor, reduciendo el tamaño de la carga útil (*payload*). Si bien los formatos de serialización binaria (como Protobuf o CBOR) ofrecen una mayor eficiencia en consumo de memoria y ancho de banda en microcontroladores, la adopción de JSON en este proyecto proporciona un balance ideal entre una menor verbosidad frente a XML y una integración nativa de alta interoperabilidad con el Dashboard web y el broker MQTT [12].

En el sistema de transmisión, las cargas útiles en formato JSON se encapsulan tanto en tramas MQTT como en paquetes WebSocket para su transporte directo hacia la nube.

1.8. Integración Web y Monitoreo en Tiempo Real

La capa de visualización del sistema de transmisión trasciende los portales web estáticos mediante la implementación de tecnologías de flujo de datos continuo. El uso de protocolos de baja latencia garantiza que los estados de la impresora fiscal se reflejen en el Dashboard con un retraso imperceptible, cumpliendo con los estándares de monitoreo industrial en tiempo real [8].

1.8.1. Protocolo WebSockets

El protocolo **WebSocket**, definido en el RFC 6455, proporciona canales de comunicación **full-duplex** sobre una única conexión TCP [8]. A diferencia del protocolo HTTP, que es de naturaleza *half-duplex* y carece de estado, WebSocket permite que el servidor envíe información al cliente de forma proactiva sin esperar una solicitud previa.

El establecimiento de la comunicación se rige por el proceso **Handshake**, el cual inicia como una solicitud HTTP convencional. Durante este intercambio, el cliente y el servidor acuerdan una "actualización de protocolo" (*Protocol Upgrade*), transformando la conexión HTTP en un flujo de datos binarios bidireccional y persistente. Una vez establecida, se eliminan la cabeceras (*headers*) repetitivas de HTTP, lo que reduce drásticamente el payload de la conexión y permite una transferencia de datos más eficiente y con menor latencia en comparación con técnicas como el *long polling* [10].

1.8.2. MQTT sobre WebSockets

En la arquitectura final de este proyecto, se implementó la técnica de **túnel o encapsulamiento de MQTT sobre WebSockets**. Esta integración es fundamental debido a que los navegadores web actuales no permiten la apertura de conexione TCP crudas (*raw sockets*) necesarias para el protocolo MQTT nativo [12, 10].

Bajo este esquema, la conexión WebSocket actúa como una tubería externa o "sobre" para los paquetes MQTT. El broker HiveMQ coloca cada paquete MQTT (como

CONNECT o PUBLISH) dentro de una trama binaria de WebSocket para enviarla al Dashboard, donde la librería de cliente JavaScript (MQTT.js) desaempaqueta la información y la procesa como una trama de red estándar [12]. En el sistema de transmisión, esta comunicación segura se realiza a través del **puerto 8884**, lo que permite atravesar firewalls corporativos que habitualmente bloquean los puertos estándar de MQTT (1883/8883) pero permiten el tráfico web convencional. Esta arquitectura desacoplada garantiza la **resilencia cibernética** del sistema, permitiendo un monitoreo directo entre el navegador y el broker sin intermediarios adicionales.

1.9. Arquitectura de Gestión y Orquestación: Framework Django

La capa superior del ecosistema de transmición requiere de un motor robusto capaz de centralizar la información, gestionar la seguridad de acceso y servir de puente entre el almacenamiento de datos y la interfaz de usuario. Para este propósito, se seleccionó **Django**, un framework web de alto nivel basado en el lenguaje de programación **Python** [4]. Python se caracteriza por ser un lenguaje interpretado de propósito general, cuya filosofía hace hincapíe en la legibilidad del código y la eficiencia técnica, características esenciales para el mantenimiento de sistemas industriales a largo plazo.

1.9.1. Patrón de Diseño MVT (Model-View-Template)

Django fundamenta su operación en una arquitectura denominada **Model-View-Template**, la cual permite una separación clara de responsabilidades en el servidor de aplicaciones [4]:

- **Modelo (Model):** Representa la fuente de información definitiva de los datos. En este proyecto, el modelo define la estructura de las tablas donde se gestionan los equipos fiscales y los registros de auditoría.
- **Vista (View):** Actúa como el mediador lógico que procesa las solicitudes de los usuarios y retorna las respuestas adecuadas. Es el componente encargado de la orquestación del sistema.
- **Plantilla (Template):** Es la capa de presentación que define cómo se muestran los datos al usuario final, integrando el contenido generado por el servidor con el Dashboard web.

1.9.2. Arquitectura Web Eficiente: HTMX y la comunicación Asíncrona

En la evolución del desarrollo web para sistemas embebidos, ha surgido una dicotomía entre las aplicaciones tradicionales multipágina (MPA) y las aplicaciones de una sola página (SPA) basadas en pesados *frameworks* de JavaScript. HTMX se presenta como una alternativa que permite extender las capacidades del HTML para realizar peticiones AJAX, WebSockets y Server-Sent Events directamente desde los atributos del mercado [9].

La relevancia de HTMX en este proyecto radica en:

- **Reducción de la Carga en el Cliente:** Al procesar la lógica de presentación en el servidor (Django) y enviar solo fragmentos de HTML al navegador, se minimiza la necesidad de procesar grandes archivos de *scripts* en el dispositivo del usuario final.
- **Reactividad sin Complejidad:** Permite actualizaciones parciales del Dashboard web (como el estado de conexión de la impresora) en tiempo real, manteniendo la simplicidad del protocolo HTTP sin la sobrecarga de un motor de renderizado del lado del cliente.

1.9.3. Gestión Administrativa y Persistencia

Una de las características de este framework es su capacidad de generar dinámicamente una **interfaz administrativa** (Admin Panel). Esta herramienta permite que los administradores del sistema en **The Factory HKA** realicen operaciones **CRUD** (Crear, Leer, Actualizar y Borrar) sobre la base de datos de los equipos de forma segura, sin necesidad de interactuar directamente con el motor de base de datos SQL [4].

En la arquitectura del dispositivo de transmisión, Django cumple el rol de Servidor de Aplicaciones, diferenciándose de un servidor web estático al procesar lógica compleja y garantizar que solo usuarios autenticados puedan acceder al monitoreo en tiempo real de los registros fiscales. Esta robustez, sumada a su enfoque en la reutilización de componentes y el principio *don't repeat yourself* (DRY), asegura que el sistema sea escalable ante el crecimiento del parque de impresoras instaladas [4].

CAPÍTULO 2

DESARROLLO

2.1. Diseño Conceptual de la Arquitectura IIoT

El diseño conceptual del Módulo de Transmisión IoT (MT-IoT) se fundamenta en la necesidad de transformar equipos de control fiscal, tradicionalmente aislados, en nodos inteligentes capaces de integrarse a una red de monitoreo centralizada. Tal como se estableció en el plan de trabajo durante las semanas 3 y 4, el objetivo fue definir los planos técnicos y lógicos para posicionar al microcontrolador ESP32 como un puente (*Gateway*) entre el hardware fiscal y la nube [17, 2].

El desarrollo del proyecto se organizó en cuatro fases metodológicas sucesivas, cuyo flujo general se resume en la Figura 2.1:

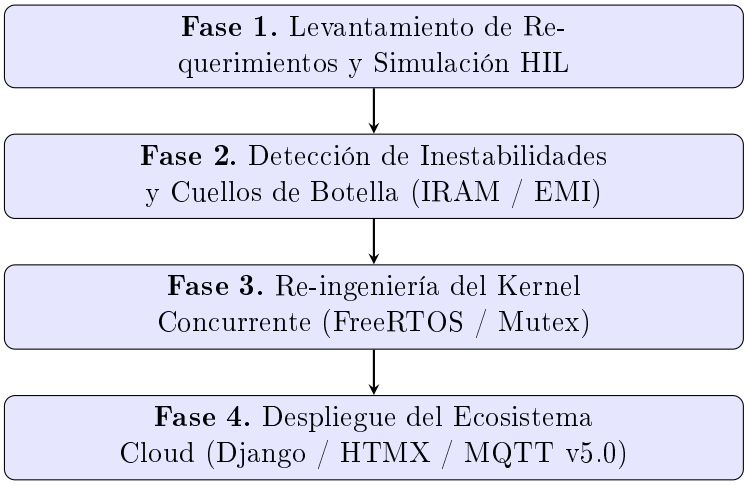


Figura 2.1. Flujo metodológico general del trabajo de pasantía organizado en cuatro fases sucesivas.

Para garantizar la escalabilidad y robustez del sistema (véase la Figura 2.2), se optó por una arquitectura distribuida de tres niveles, alineada con los estándares industriales

del Internet de las Cosas (IIoT) [17, 2]:

- **Nivel de Borde (*Edge*):** Representado por la impresora fiscal HKA80. En esta capa se origina la información crítica mediante registros de estado (**Status S1**) y diagnóstico (**SV2**) [14]. La captura de estos datos se realiza a través de una interfaz de comunicación síncrona dedicada (operando a una frecuencia de reloj de 100 kHz), diseñada para funcionar de forma transparente sin interferir con las operaciones de impresión. Esta transparencia se logra manteniendo un intervalo de sondeo (*polling*) estrictamente acotado, asegurando que los tiempos de respuesta de cada transacción no excedan los umbrales temporales tolerados por el firmware fiscal homologado del equipo; premisa que será demostrada posteriormente mediante un perfilado de tiempos en el bus de datos.

- **Nivel de Pasarela (Gateway):** Ejecutado por el Módulo de Transmisión IoT (basado en el SoC ESP32). Su función es doble: actúa como maestro en la interfaz local con el periférico y como cliente seguro (*Publisher*) hacia la red Wi-Fi [7]. En este nivel se implementa la lógica de empaquetado de datos en formato JSON y la gestión de seguridad mediante TLS/SSL, permitiendo el acceso a registros críticos almacenados en la Memoria de Trabajo y Memoria de Auditoría sin interferir con el cabezal de impresión.

- **Nivel de Gestión y Visualización (Cloud):** Compuesto por el *Broker* HiveMQ y el servidor de aplicaciones desarrollado en Django [4]. Este nivel orquesta la recepción asíncrona de telemetría y su visualización en tiempo real en el Dashboard mediante el transporte de tramas *MQTT* sobre un túnel *WebSocket* (WSS) [12, 8].

La elaboración de los diagramas técnicos (Figura 2.2 y Figura 2.3) respondió a la necesidad de visualizar el flujo de datos desde el comando inicial `0x05h` (`ENQ`) enviado por el MT-IoT a través de la interfaz serie asíncrona RS-232, hasta el desempaquetado (*Data Unpackaging*) en la plataforma web para una fase inicial y conceptual del proyecto [2]. El diseño de la arquitectura de comunicaciones fue especialmente crítico para definir el *Topic Tree* (jerarquía de tópicos), el uso de *User Properties* de MQTT v5.0, y el modelo de seguridad mediante el encapsulamiento sobre TLS/SSL [12]. La inyección de metadatos (como la versión del firmware y la intensidad de señal Wi-Fi) directamente en las *User Properties* de la cabecera del protocolo evita sobrecargar el cuerpo del mensaje JSON, lo cual reduce el consumo de ancho de banda y elimina la necesidad de procesamiento o *parsing* adicional en el cliente web suscriptor [12].

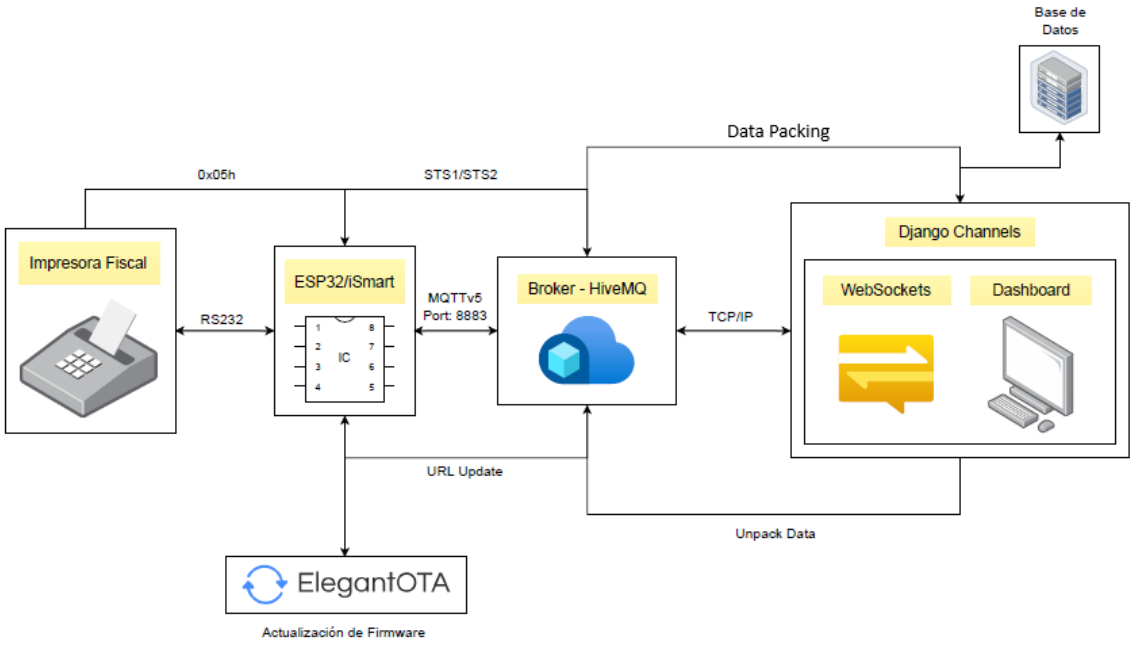


Figura 2.2. Diagrama de la Arquitectura IIoT distribuida en tres niveles (Edge, Gateway y Cloud).

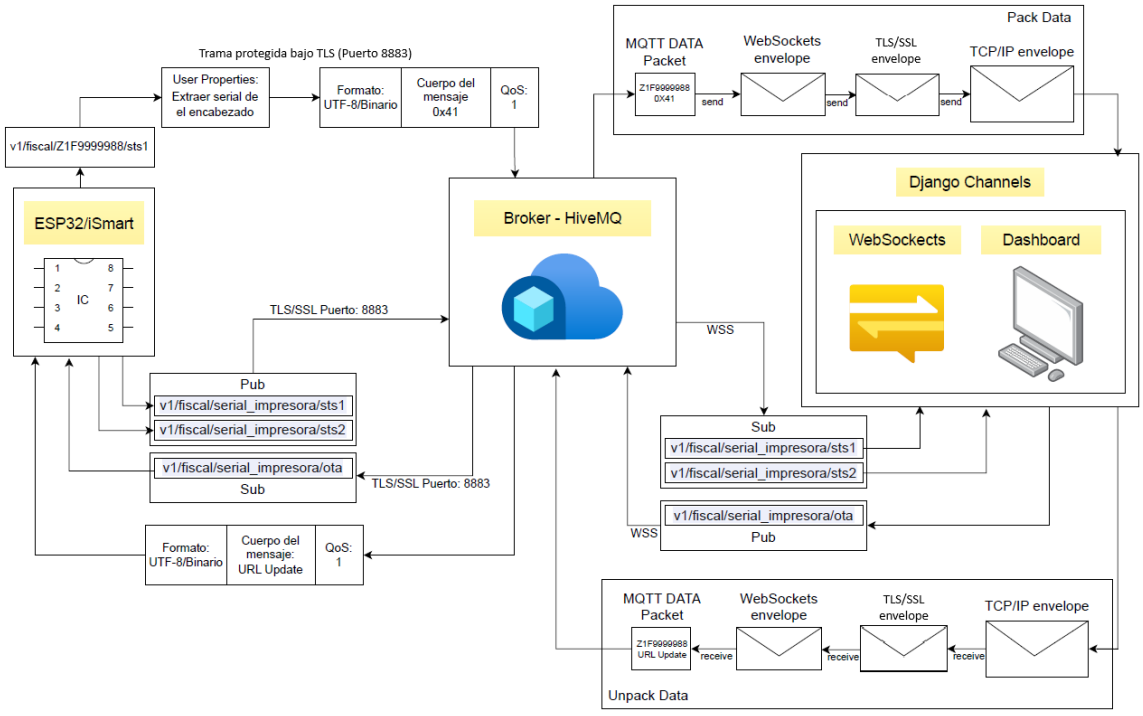


Figura 2.3. Diagrama propuesto para la arquitectura de comunicaciones MQTT.

2.2. Análisis de Decisiones de Arquitectura y Compromisos de Ingeniería

2.2.1. Microcontrolador de Doble Núcleo: ESP32

Para la implementación del nodo de transmisión, se seleccionó la tarjeta de desarrollo ESP32 DevKitC V4. Esta elección se fundamentó en la necesidad de gestionar procesos concurrentes industriales [7, 1]. En la arquitectura final, se asignaron tareas específicas a cada núcleo para garantizar el determinismo: mientras un núcleo gestiona el sondeo (*polling*) de la impresora fiscal cada 5 segundos, el segundo núcleo administra la pila de red y la mensajería MQTT v5.0, evitando que las latencias de internet bloqueen la captura de datos locales.

Durante el prototipado, se identificó una limitación crítica en el hardware relacionada con la asignación de periféricos. Al intentar utilizar el UART1 en sus pines por defecto (GPIO 9 y 10), el sistema experimentaba reinicios constantes (bootloops) debido a que estos pines están anclados al bus SPI de la memoria Flash interna del SoC [7]. Para solventar esta colisión eléctrica, se reconfiguró la matriz de señales, moviendo la interfaz de comunicación fiscal hacia los pines libres GPIO 18 (TX) y GPIO 19 (RX), logrando una transmisión estable (véase la Figura 2.4).

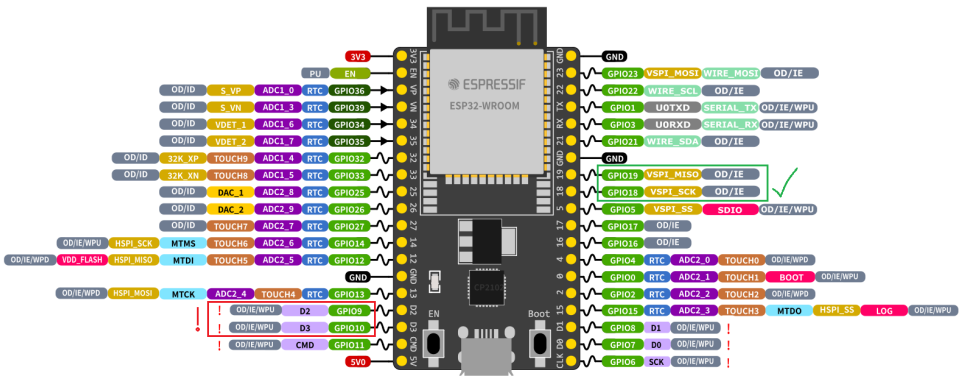


Figura 2.4. Pinlayout de la tarjeta ESP32 DevKitC V4 y reasignación de pines seriales.

La gestión de la memoria interna representó el mayor desafío técnico. Con la implementación completa de la pila TLS/SSL y el empaquetado dinámico de mensajes JSON, el uso de la memoria IRAM alcanzó un nivel crítico del 78.39 % en la semana 6. Tras un análisis de perfilado del *firmware*, se aplicó una reingeniería de recursos mediante la herramienta

`menuconfig` desactivando la aceleración IRAM para el *stack* Wi-Fi (véase la Figura 2.5). Esta decisión redujo el consumo de memoria a un 52.26 % (un ahorro directo del 26.13 %), liberando el *heap* necesario para alojar el búfer de actualización remota (OTA).

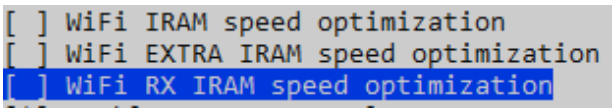


Figura 2.5. Configuración aplicada en menuconfig para optimización de IRAM.

Finalmente, para optimizar el comportamiento térmico y garantizar la estabilidad del hardware en operación continua 24/7 dentro de un gabinete comercial, se ajustó la frecuencia del CPU a 80 MHz mediante la herramienta `menuconfig` (vease Figura 2.6) [6].Esta frecuencia de reloj mínima operativa demostró ser suficiente para sostener de forma determinista el procesamiento del sistema y la aceleración criptográfica de AES por hardware requerida por la conexión segura hacia HiveMQ, evitando la disipación de calor innecesaria asociada a frecuencias superiores (160/240 MHz).

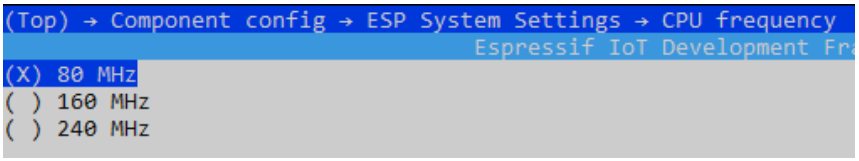


Figura 2.6. Ajuste de frecuencia de reloj del CPU a 80 MHz en `menuconfig` para estabilidad térmica en operación continua.

2.2.2. Unidad de Control Fiscal: Impresora HKA80

La impresora fiscal HKA80 (véase la Figura 2.7) actúa como el activo de borde (*Edge*) central del sistema. Su selección se fundamentó en su arquitectura modular, la cual permite el acceso a registros críticos almacenados en su Memoria de Trabajo y Memoria de Auditoría sin interrumpir mecánicamente al cabezal de impresión.

Para la integración con el MT-IoT, se definieron los parámetros obligatorios del protocolo: tasa de transferencia de 9600 bps, longitud de 8 bits, paridad par y 1 bit de parada [14]. La fase inicial de la telemetría priorizó el registro `Status S1` (identidad del equipo y acumuladores) y el comando `SV2` (versión de *firmware* del periférico).



Figura 2.7. Impresora fiscal modelo HKA80 autorizada por la normativa vigente.

Un hallazgo crítico durante el desarrollo del **analizador sintáctico (*parser*)** fue la identificación de un carácter invisible de salto de línea (`\n`) inmediatamente antes del fin de trama (ETX) emitido por la impresora. Este factor desplazaba los punteros de memoria provocando errores de tipo *Off-By-One*, los cuales corrompían la carga útil (*payload*) MQTT. La anomalía fue resuelta introduciendo un ajuste dinámico que delimita el término de la cadena mediante la inyección explícita del carácter nulo (`\0`) en el *buffer* de salida, asegurando la integridad de los datos extraídos. La secuencia analítica de esta corrección se describe en el Algoritmo 2.1, mientras que su implementación en lenguaje C se conserva en el Apéndice A.

Esta sanitización de datos asegura un determinismo absoluto en la captura de registros cada 5 segundos, permitiendo que el MT-IoT actúe como un *Gateway* transparente que no añade latencia a la red transaccional del contribuyente.

2.2.3. Justificación del Framework Nativo: Determinismo vs. Latencia (ESP-IDF)

Durante la semana de definición tecnológica se realizó un análisis comparativo entre el entorno Arduino IDE y el Espressif IoT Development Framework (ESP-IDF) (véase la

Algoritmo 2.1. Mitigación analítica del error *Off-By-One* en el procesamiento de la trama SV2.

Recibir el bloque de respuesta y validar su redundancia (LRC).
Calcular la posición del terminador: $p_{ETX} \leftarrow n - 2$.
Estimar el índice inicial del campo de versión: $i_0 \leftarrow p_{ETX} - L_{fw} - 1$.
if $i_0 \geq 1$ **then**
    Reservar/bloquear la sección crítica de memoria.
    Copiar L_{fw} caracteres desde i_0 hacia el *buffer* del campo de versión.
    Asignar el carácter terminador nulo ("\0") en la posición L_{fw} del *buffer* de destino.
    Liberar la sección crítica.
    return verdadero
end if
return falso

Tabla 2.1).

Tabla 2.1. Análisis comparativo de entornos de desarrollo para el nodo Gateway.

Criterio	**Arduino IDE**	**ESP-IDF v5.5.2**
Modelo de Ejecución	Bucle secuencial único (`setup/loop`).	Multitarea concurrente (`app_main`).
Gestión de Tareas	Dependiente de bloqueos (`delay`).	Planificación RTOS (`xTaskCreate`).
Gestión de Memoria	Compilación estandarizada, alta redundancia.	Control sobre componentes vía `menuconfig`.
Idoneidad	Prototipado rápido, baja escalabilidad.	Despliegue industrial y seguridad TLS.

La elección del framework de desarrollo representó un compromiso (*trade-off*) crítico entre velocidad de prototipado y determinismo temporal. Aunque el ecosistema Arduino permitía un despliegue ágil, su modelo de ejecución mono-hilo imponía latencias inaccesibles. En un entorno transaccional fiscal, un retardo provocado por un `delay()` bloqueante en el stack Wi-Fi arriesgaba sobrepasar la ventana temporal de lectura del firmware fiscal homologado. Por ello, se justificó el uso nativo de **ESP-IDF**, permitiendo gestión explícita de memoria en C y la asignación determinista de tareas mediante FreeRTOS.

Si bien Arduino ofrece una curva de aprendizaje reducida, su modelo de ejecución

secuencial representa un cuello de botella sistémico. Al optar por ESP-IDF [5], la migración del punto de entrada del programa hacia `app_main` permitió el despliegue de la API de FreeRTOS. A través de la función `xTaskCreatePinnedToCore` [1], se logró orquestar tareas independientes fijadas a núcleos específicos, garantizando que el *stack* de red no interrumpiera la captura local por UART.

Adicionalmente, el sistema de construcción CMake permitió una declaración explícita de componentes (editando el archivo `CMakeLists.txt`, véase la Figura 2.8), asegurando que solo se incluyeran dependencias estrictamente necesarias como `esp_https_ota` y `mqtt`. Estas decisiones de ingeniería permitieron que el ESP32 operara con una arquitectura modular y determinista.

```
PRIV_REQUIRES esp_wifi nvs_flash
REQUIRES driver esp_driver_uart mqtt esp_wifi esp_event esp_http_client esp_https_ota
INCLUDE_DIRS ".")
```

Figura 2.8. Inclusión estricta de dependencias en el archivo de construcción CMakeLists.txt.

2.3. Fase de Prototipado y Simulación Inicial

El desarrollo de la primera fase del sistema de transmisión comenzó con un enfoque de validación incremental, motivado principalmente por la indisponibilidad inicial de la interfaz de desarrollo física de la impresora fiscal corporativa [15]. Para mitigar este cuello de botella y asegurar el progreso del cronograma técnico, se diseñó una metodología de simulación por software que permitió emular el comportamiento del hardware de borde (*Edge*) y verificar los cimientos del *firmware* en un entorno controlado antes de la integración física final [15].

2.3.1. Configuración de la Comunicación Asíncrona (UART)

La comunicación física inicial entre el MT-IoT y la unidad fiscal se estructuró bajo el estándar asíncrono RS-232, utilizando los parámetros serie nativos del hardware [14]. Al no contar con el periférico real en el laboratorio, se empleó un computador personal como activo de simulación, lo que requirió la estructuración de dos entornos de prueba sucesivos:

1. **Pruebas con Terminales de Datos (RealTerm):** En primera instancia, se intentó inyectar y monitorear tramas de datos binarios de forma manual. Si bien este método permitió verificar respuestas básicas al comando *Enquiry* (`ENQ = 0x05h`), tal como se aprecia en la Figura 2.9, la herramienta resultó inviable por conflictos de codificación: la consola presentaba limitaciones para concatenar de forma determinista caracteres hexadecimales puros (como los terminadores `STX 0x02` y `ETX 0x03`) con los 129 bytes de texto ASCII requeridos para emular una respuesta válida del `Status S1` [14].

2. **Simulador Dedicado en Python:** Para solventar el problema de codificación mixta, se programó un *script* de automatización nativo utilizando la biblioteca `pyserial`. Este programa simuló de manera fidedigna los registros de la impresora (a 9600 bps, 8 bits, paridad par y 1 bit de parada), interceptando solicitudes seriales y estructurando dinámicamente respuestas válidas, tal como se evidencia en la máquina de estados de la Figura 2.10 (el listado del programa se conserva en el Apéndice A).

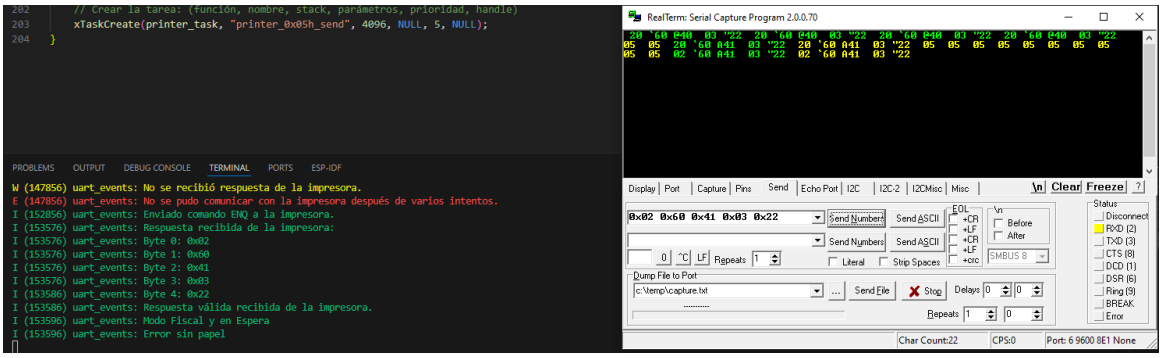


Figura 2.9. Captura de inyección manual de tramas y comando ENQ mediante la terminal de datos RealTerm.

En el entorno del MT-IoT, bajo el núcleo del RTOS, se implementó la tarea base denominada `printer_polling_task` [1]. El aspecto clave de esta rutina fue la sustitución de los retardos secuenciales bloqueantes por una cesión voluntaria del control al planificador (véase el Algoritmo 2.2) [1]. Al utilizar la macro `pdMS_TO_TICKS`, el planificador suspende la tarea durante la ventana de 5000 ms, liberando tiempo de CPU para que el segundo núcleo procese el *stack* de red sin interrupciones [1].

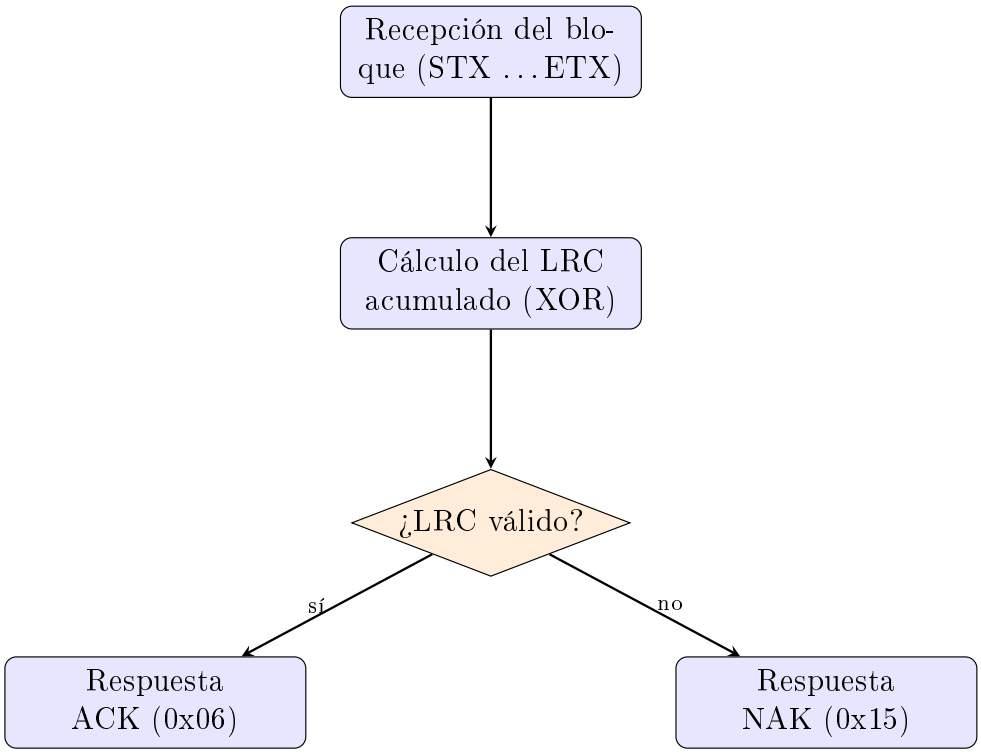


Figura 2.10. Máquina de estados del simulador fiscal en software.

Algoritmo 2.2. Tarea de interrogación serial no bloqueante.

```
Inicializar la tarea de sondeo fiscal.
while verdadero do
    Transmitir el carácter de encuesta ENQ (0x05) por el bus local.
    Ceder el control al planificador durante T_s = 5000 ms.
end while
```

2.3.2. Integración de Mensajería Ligera (MQTT sobre TCP)

Una vez consolidada la captura de datos locales, el desarrollo se orientó a la capa de transporte. Previo a la implementación final, se realizó una validación enviando tramas crudas mediante peticiones `HTTP GET` hacia la nube, lo que confirmó el correcto dimensionamiento del *heap* para el *stack* de red Wi-Fi de ESP-IDF [5].

Sin embargo, para cumplir con los requerimientos industriales de mínima sobrecarga (*overhead*) de cabecera y envío asíncrono, se migró hacia el estándar MQTT v5.0 sobre conexiones TCP/IP [12]. El prototipado inicial utilizó el *broker* de código abierto *Eclipse Mosquitto* en el entorno local (puerto estándar 1883 no cifrado) [12].

El *firmware* gestionó la conexión mediante el bucle de eventos `mqtt_event_handler`. Al recibir la confirmación de conexión (`MQTT_EVENT_CONNECTED`), el sistema disparaba la tarea de publicación a través del nivel de calidad de servicio QoS 1 (entrega garantizada al menos una vez). Las pruebas operativas se realizaron empaquetando el **Status S1** simulado en cadenas JSON planas hacia una jerarquía preliminar de tópicos, sentando las bases para el procesamiento asíncrono de eventos y los mecanismos de persistencia del servidor web final [4].

La efectividad de este canal de transporte se validó mediante la intercepción del *payload* en el *Broker* HiveMQ. Tal como se documenta en la Tabla 2.2, el microcontrolador logró empaquetar exitosamente los registros crudos de la impresora en una estructura clave-valor legible, confirmando la correcta extracción del RIF, los acumuladores de ventas y los estados operativos. El listado textual del mensaje se conserva en el Apéndice A.

Tabla 2.2. Estructura del mensaje JSON del `Status S1` publicado hacia el *broker* MQTT.

Clave	**Tipo**	**Descripción**
`timestamp`	Entero	Marca temporal del RTOS en milisegundos
`sts1`	Cadena	Estado lógico del equipo (p. ej., `0x60`)
`sts2`	Cadena	Estado del mecanismo de impresión (p. ej., `0x40`)
`estado`	Cadena	Descripción textual del estado lógico
`error`	Cadena	Descripción textual del error mecánico
`cajero`	Cadena	Código o identificador del operador de caja
`ventas`	Cadena	Acumulador de ventas diarias
`fact_emitidas`	Cadena	Contador de facturas emitidas
`rif`	Cadena	Registro de Información Fiscal del contribuyente
`registro`	Cadena	Número de registro de la máquina

2.4. Diagnóstico e Identificación de Inestabilidades en Entornos Industriales Hostiles

A medida que el *firmware* del Módulo de Transmisión IoT (MT-IoT) incorporó las capas de seguridad criptográfica y la lógica de transporte hacia la nube, el sistema comenzó a exhibir cuellos de botella operativos y fallos de estabilidad. Esta sección analiza las limitaciones críticas identificadas en el hardware y el software durante las pruebas de estrés en condiciones operativas, las cuales comprometían el determinismo y la alta disponibilidad exigidas para un dispositivo industrial [2].

2.4.1. Restricciones de Memoria y Gestión de la pila IRAM

La integración de una comunicación segura basada en cifrado de clave pública hacia el *Broker* requirió la habilitación de la pila de seguridad Mbed TLS nativa de ESP-IDF [5]. Sin embargo, la inicialización del *handshake* TLS/SSL, la validación de certificados raíz para HiveMQ Cloud y el procesamiento dinámico de objetos mediante la librería `cJSON` generaron un consumo exponencial de memoria en el microcontrolador [1].

En la arquitectura del SoC ESP32, la memoria interna SRAM de 520 KB se divide lógicamente en segmentos de instrucción (IRAM) y de datos (DRAM) [7]. Durante las pruebas de estrés con la pila segura y el cliente de telemetría activos, el uso de la IRAM escaló críticamente hasta alcanzar el 78.39 % (102,751 Bytes utilizados de 131,072 Bytes disponibles), dejando un estrecho margen operativo, tal como se evidencia en la Figura 2.11.

Este escenario de alta ocupación presentaba riesgos técnicos severos:

- **Desbordamiento de Pila (*Stack Overflow*):** El planificador de tareas de FreeRTOS quedaba vulnerable ante picos de tráfico de red o lazos de reconexión asíncronos, donde cualquier asignación de pila adicional provocaría un reinicio crítico del sistema por pánico del *kernel*.
- **Inviabilidad del Sistema de Actualización (OTA):** La saturación de memoria impedía reservar el espacio dinámico (*heap*) requerido para alojar los *buffers* de descarga del cliente HTTP, imposibilitando la orquestación de la tabla de particiones dual (Factory + 2 OTA).

Memory Type Usage Summary

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	651014			
.text	651014			
Flash Data	199836			
.rodata	199580			
.appdesc	256			
IRAM	102751	78.39	28321	131072
.text	101723	77.61		
.vectors	1028	0.78		
DRAM	37852	20.94	142884	180736
.bss	21688	12.0		
.data	16164	8.94		
RTC SLOW	56	0.68	8136	8192
.force_slow	32	0.39		
.rtc_slow_reserved	24	0.29		

Figura 2.11. Perfilado de memoria revelando un nivel crítico de ocupación de la IRAM (78.39 %).

2.4.2. Interferencia Electromagnética y Basura en el Bus UART

En la etapa de prototipado inicial, la captura local de datos fiscales se realizaba utilizando el transceptor asíncrono UART [14]. Al implementar el flujo bidireccional estricto gobernado por caracteres de control físicos (`ACK 0x06h` y `NAK 0x15h`), se evidenció que este estándar presentaba vulnerabilidades inaceptables para la robustez industrial.

Debido a la ausencia de blindaje en el cable de interconexión local y la naturaleza asíncrona del bus RS-232, la línea de transmisión presentó vulnerabilidad a perturbaciones en la señal. Esta condición generaba corrupción de tramas a nivel del controlador UART del microcontrolador, evidenciándose mediante la recepción de bytes anómalos (*garbage data*) y errores de enmarcado (*Framing Errors*). Dichas anomalías desincronizaban los punteros de lectura del *buffer*, inyectando caracteres no imprimibles en el flujo de entrada previo a su procesamiento.

Al alterarse las tramas físicas (especialmente los 129 bytes del comando `Status S1`), el *firmware* fallaba en la comprobación matemática de redundancia longitudinal (LRC) [14]. Para mitigar temporalmente este fenómeno antes de la migración definitiva a la interfaz síncrona, se implementó una estricta validación por *software* que descartaba las tramas corruptas y liberaba el bus, tal como se documenta en el Algoritmo 2.3.

Algoritmo 2.3. Validación de integridad de trama e intercepción de ruido electromagnético.

1: Recibir la trama (STX, D, ETX, LRC_{rx}).
2: **if** $STX \neq$ `0x02` **o** $ETX \neq$ `0x03` **then**
3: Enviar `NAK` y descartar la trama.
4: **end if**
5: Calcular $LRC_{calc} \leftarrow \bigoplus_{i=1}^{n-1} b_i$.
6: **if** $LRC_{rx} = LRC_{calc}$ **then**
7: Enviar `ACK`.
8: **else**
9: Enviar `NAK`, incrementar el contador de errores y liberar el bus.
10: **end if**

2.4.3. Problemas de Concurrencia (Race Conditions)

La administración multitarea de FreeRTOS permite ejecutar procesos de forma paralela en los dos núcleos del ESP32 [1]. Sin embargo, la coexistencia de la tarea de adquisición fiscal continua y la recepción de comandos inalámbricos introdujo condiciones de carrera (*Race Conditions*) severas:

1. **Colisiones en Periféricos Compartidos (*Hardware Lock*):** Durante las pruebas iniciales de actualización, si el microcontrolador recibía un comando de descarga OTA por MQTT exactamente en el mismo instante en que la tarea fiscal iniciaba su ciclo de sondeo, ambas rutinas intentaban acceder simultáneamente al controlador del bus de comunicación. Al carecer de exclusión mutua coordinada mediante un semáforo o *mutex* de FreeRTOS, el sistema generaba un fallo de aserción (`assert failed`) forzando el reinicio del SoC (véase la Figura 2.12).

2. **Inconsistencia de Datos por Concurrencia (*Race Condition*):** A nivel lógico, la falta de control de acceso síncrono permitía que la tarea de monitoreo actualizara los búferes de ventas mientras la tarea MQTT leía de forma asíncrona dichos campos para empaquetar el *payload* JSON. Esto ocasionaba la lectura de tramas híbridas o incompletas en la memoria intermedia, comprometiendo la integridad de la telemetría transmitida.

Estas limitaciones demostraron empíricamente que, para un entorno de control tributario, no basta con lograr comunicación bidireccional; es mandatorio implementar mecanismos de exclusión mutua gestionados por el RTOS (tales como *Mutexes*) para arbitrar el acceso concurrente a los recursos de *hardware* compartidos y garantizar optimizaciones granulares de la memoria [7].

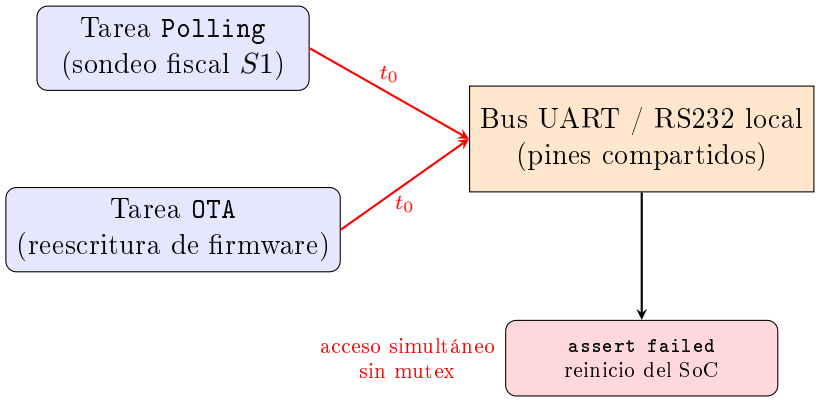


Figura 2.12. Traza de depuración y registro de evento ante colisión por acceso concurrente al bus local (Choque de Trenes).

2.5. Diseño de un Kernel Concurrente e Invariante de Red a Prueba de Condiciones de Carrera

Una vez identificados los cuellos de botella en la gestión de memoria, la vulnerabilidad al ruido eléctrico de la línea de datos asíncrona y las condiciones de carrera en los hilos de FreeRTOS, se procedió a realizar una re-ingeniería profunda sobre los cimientos del *firmware* del MT-IoT [5]. Esta fase se enfocó en reestructurar la capa de abstracción de *hardware* y optimizar el planificador de tareas de tiempo real para asegurar el determinismo y la resiliencia del sistema en entornos industriales hostiles.

2.5.1. Implementación de la Interfaz Síncrona de Alto Rendimiento

Para mitigar la inyección de bytes corruptos generada por la susceptibilidad electromagnética del estándar asíncrono, se migró la interconexión física local hacia una interfaz de comunicación serie síncrona dedicada, operando a una frecuencia de 100 kHz [7]. A diferencia de la comunicación UART anterior, este método basa el muestreo de datos en una señal de temporización (*Clock*) dedicada, generada por el ESP32 en rol de maestro, eliminando de raíz las desviaciones por deriva de reloj (*baud-rate drift*) y garantizando una transmisión determinista [5]. La disposición física de este enlace maestro-esclavo se ilustra en la Figura 2.13.

Para blindar la integridad lógica de las tramas tributarias en esta nueva capa física, se implementó una rutina matemática de validación de redundancia longitudinal (LRC). Tras recibir cada bloque de datos, el *firmware* ejecuta un lazo cerrado que realiza opera-

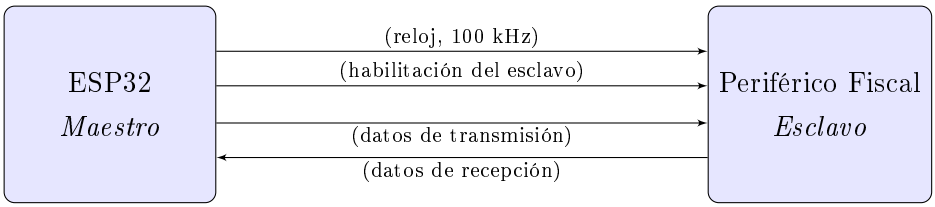


Figura 2.13. Interacción maestro-esclavo sobre el bus síncrono local, con señal de reloj dedicada a 100 kHz y vías independientes de transmisión y recepción.

ciones lógicas `XOR` acumulativas sobre los bytes recibidos, contrastando el resultado con el *checksum* enviado por el periférico fiscal [14].

Si la trama es declarada válida, el MT-IoT responde con un carácter `ACK` (`0x06h`); de lo contrario, envía un `NAK` (`0x15h`) y procede de inmediato a vaciar (*flush*) el *buffer* de recepción de entrada y reiniciar sus punteros de lectura, descartando la secuencia anómala e impidiendo la propagación de datos corruptos hacia la memoria del sistema.

Adicionalmente, para evitar bloqueos del procesador si el periférico es apagado físicamente, se establecieron temporizadores (*timeouts*) estrictos. Si se supera el límite de fallos continuos, el *firmware* transiciona al estado de pánico y activa una máquina de reconexión, sin bloquear las tareas de red [1].

2.5.2. Orquestación Concurrente mediante FreeRTOS

El control cooperativo del MT-IoT se organizó bajo el *kernel* de FreeRTOS mediante un esquema de exclusión mutua de dos niveles, solventando la condición de carrera identificada en la etapa de prototipado:

1. **Exclusión Mutua de Datos en RAM (`mqtt_data_mutex`):** Se implementó un semáforo tipo *Mutex* para proteger la estructura `mqtt_data_t` en la DRAM. Este candado lógico garantiza que la tarea de monitoreo local no pueda sobrescribir variables de facturación mientras la tarea MQTT está leyendo el bloque para formatear el *payload* JSON, evitando el tránsito de datos híbridos.

2. **Exclusión Mutua del Bus Físico (`bus_local_mutex`):** Para resolver la colisión eléctrica local (choque de trenes) cuando una actualización remota coincide con el sondeo periódico de 5 segundos, se introdujo un segundo *Mutex* de *hardware*. Como se observa en la Figura 2.14, tanto la tarea de sondeo como la rutina OTA deben

competir y adquirir con éxito este semáforo antes de poder energizar el canal de comunicación físico. El listado de la implementación se conserva en el Apéndice A.

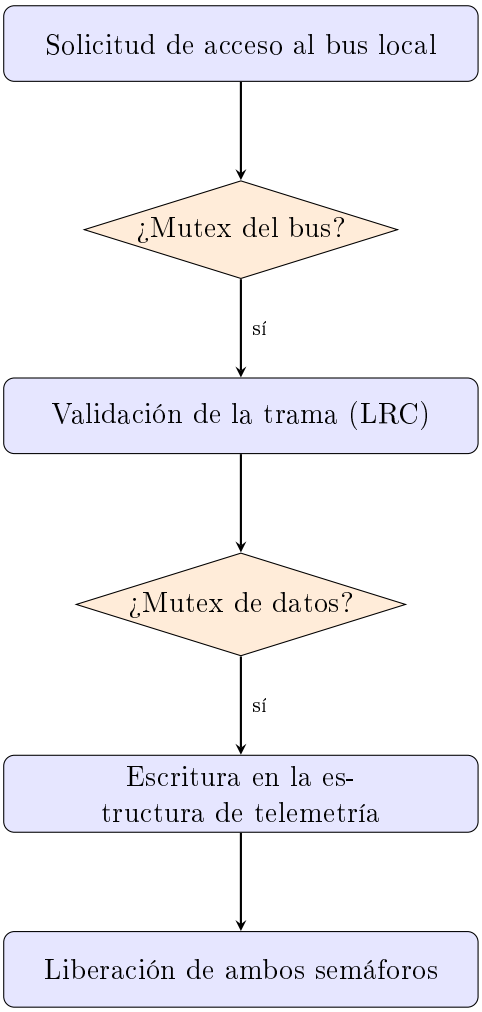


Figura 2.14. Exclusión mutua en dos niveles para la protección de la memoria RAM y del bus físico.

Para proteger la memoria (*heap*) ante ráfagas de comandos MQTT, la lógica OTA se desacopló del bucle principal. Se creó la tarea `ota_supervisor_task`. Al detectarse un comando de actualización, el manejador MQTT empuja la solicitud a una cola segura (`ota_cmd_queue`). Al despertar, el supervisor activa una bandera global de "freno de mano" (`ota_en_progreso = true`) que paraliza la tarea fiscal, y ejecuta la descarga en total aislamiento [5].

2.5.3. Estrategias de Resiliencia de Red y DNS Injection

Para cumplir con los requerimientos de robustez industrial, el *firmware* se dotó de mecanismos de autodefensa jerárquicos. A nivel de enlace, ante la pérdida de la conexión con el punto de acceso (AP), la capa de red del microcontrolador gatilla asíncronamente el evento `WIFI_EVENT_STA_DISCONNECTED`, cuyo campo `reason` codifica el motivo de la desconexión. El manejador responde con `esp_wifi_connect()`, iniciando un lazo de reconexión infinito que reanuda el enlace sin reiniciar la lógica de control interna [5]. La caída del servicio de internet (WAN), en cambio, se detecta en la capa de aplicación mediante los fallos de socket y los reintentos automáticos del cliente MQTT.

Uno de los aportes más significativos fue la mitigación de fallos de resolución de nombres (*DNS*) del contribuyente. Al capturar una dirección IP local a través del evento `IP_EVENT_STA_GOT_IP`, la pasarela intercepta dinámicamente la configuración de la interfaz de red (*netif*) e inyecta forzosamente los servidores principales de Google (8.8.8.8) [5]. Esta técnica asegura que el cliente TLS siempre resuelva la ruta hacia HiveMQ.

Finalmente, ante caídas temporales del servidor MQTT, el *firmware* impone un retardo de cortesía (`RECONNECT_DELAY_MS`) antes de intentar levantar el *socket* nuevamente (véase el Algoritmo 2.4). Esto evita ataques de denegación de servicio (DDoS) involuntarios contra el *Broker* y estabiliza el consumo energético del MT-IoT.

Algoritmo 2.4. Resiliencia y reconexión no bloqueante en la tarea MQTT.

1: **while** verdadero **do**
2: Esperar la notificación de un evento de la pila de red.
3: **if** la conexión con el *broker* no está activa **then**
4: Aplicar un retardo de cortesía $T_r =$ `RECONNECT_DELAY_MS`.
5: Continuar el ciclo sin descartar la telemetría pendiente.
6: **end if**
7: Publicar la telemetría retenida.
8: **end while**

2.6. Seguridad y Trazabilidad del Canal de Telemetría MQTT v5.0

Sobre el canal de telemetría MQTT establecido en la Sección 2.3.2, esta sección aborda su endurecimiento para operación en producción: la protección criptográfica del transporte

y la explotación de las capacidades avanzadas del estándar MQTT v5.0 para la trazabilidad y la detección de caídas abruptas de enlace.

2.6.1. Configuración Segura del Cliente MQTT (TLS 1.2)

El transporte de información tributaria sensible a través de internet requiere esquemas de cifrado robustos que impidan ataques de interceptación pasiva (*eavesdropping*) o inyección de contenido. Para cumplir con esta directiva, el cliente MQTT embebido en el ESP32 se configuró para establecer *sockets* seguros utilizando el protocolo TLS v1.2, apoyado en la biblioteca Mbed TLS nativa de ESP-IDF [5].

La implementación de los parámetros de seguridad se definió dentro de la estructura de configuración del cliente (`esp_mqtt_client_config_t`), aplicando dos decisiones de diseño fundamentales que se evidencian en la Tabla 2.3:

1. **Puerto Seguro y Cifrado por Hardware:** Se forzó el enrutamiento hacia el puerto estándar 8883 (`secure-mqtt`), estableciendo el transporte como `MQTT_TRANSPORT_OVER_SSL`. Esto obliga al microcontrolador a realizar un *Handshake* criptográfico, negociando algoritmos de cifrado simétrico mediante la aceleración por *hardware* integrada en el silicio del SoC [7].
2. **Autenticación mediante *Bundle* de Certificados:** Para mitigar ataques de intermediario (*Man-in-the-Middle*), el MT-IoT verifica la cadena de confianza de HiveMQ Cloud. En lugar de quemar estáticamente extensos archivos PEM en la Flash (lo que degradaría la memoria), se habilitó el parámetro `crt_bundle_attach`. Esta técnica enlaza un catálogo optimizado de Autoridades de Certificación (CAs) confiables provisto por Espressif, logrando una validación robusta con un consumo residual de almacenamiento [5].

En el extremo del servidor, el *worker* asíncrono de Django se configuró de manera homóloga para forzar el protocolo TLS v1.2, garantizando un ecosistema cerrado de extremo a extremo. Adicionalmente, el formateo del *payload* en C se implementó utilizando exclusivamente la función `snprintf` para delimitar rígidamente las cadenas de caracteres, previniendo desbordamientos de búfer (*Buffer Overflows*) y protegiendo el *stack* de la tarea de red.

Tabla 2.3. Parámetros de configuración del cliente MQTT con seguridad TLS estricta.

Parámetro	**Valor o función**
`broker.address.hostname`	Dominio del *broker* HiveMQ Cloud
`broker.address.port`	8883 (`secure-mqtt`)
`broker.address.transport`	`MQTT_TRANSPORT_OVER_SSL`
`broker.verification.crt_bundle_attach`	`esp_crt_bundle_attach` (validación de CAs)
`session.protocol_ver`	`MQTT_PROTOCOL_V_5`

2.6.2. Trazabilidad mediante User Properties y Last Will and Testament

La adopción del estándar MQTT v5.0 permitió incorporar capacidades avanzadas de diagnóstico que no eran viables en versiones anteriores (v3.1.1) [12]. El *firmware* explota estas características para optimizar la trazabilidad y la resiliencia del monitoreo en tiempo real.

En primer lugar, se implementó el uso de *User Properties* en cada paquete de publicación (`PUBLISH`). Estas propiedades actúan como metadatos en formato clave-valor (UTF-8) anidados en la cabecera del protocolo de red, permitiendo su lectura por el servidor web sin necesidad de ejecutar rutinas costosas de deserialización sobre el *payload* JSON principal [12]. Tal como se documenta en la Tabla 2.4, el *firmware* inyecta dinámicamente propiedades críticas como el RSSI (intensidad de señal Wi-Fi) y el número de serie.

En segundo lugar, para resolver la imposibilidad de realizar un sondeo constante (*polling*) desde el servidor web hacia miles de clientes, se configuró el mecanismo *Last Will and Testament* (LWT) o Testamento de MQTT [12]. Durante el establecimiento de la sesión con el *Broker*, el MT-IoT registra una "última voluntad" con un nivel de calidad `QoS 1` y la bandera de retención (*Retain Flag*) activa. El *payload* preconfigurado declara el estado `offline` y la alerta `conexion_perdida_abruptamente`; su contenido textual exacto se conserva en el Apéndice A.

Si el MT-IoT experimenta una pérdida abrupta de energía o falla de red, el *Broker* detecta la caída del *Keep-Alive* y publica el testamento automáticamente. Al interceptar

Tabla 2.4. Propiedades de usuario (*User Properties*) inyectadas en cada publicación MQTT v5.0.

Clave	**Contenido**
`modelo_dispositivo`	Modelo del equipo (`HKA80`)
`firmware_version`	Versión del firmware del MT-IoT
`serial`	Número de serie del periférico
`tipo_dato`	Clasificación del mensaje (`status_s1`)
`wifi_rssi`	Intensidad de señal Wi-Fi en dBm

este evento, el *backend* en Django ejecuta una rutina de autodefensa que congela la última lectura fiscal válida en la base de datos y emite una alerta visual, garantizando la supervisión total sin sobrecargar la red.

2.7. Arquitectura de Puente Síncrono de Reescritura Remota para Periféricos In Situ (OTA Doble Objetivo)

Uno de los requerimientos no funcionales más exigentes es la capacidad de mantener el ciclo de vida del *software* en campo sin requerir la intervención física de soporte técnico [17]. Para superar este desafío operativo, se diseñó un protocolo de Actualización Inalámbrica (OTA, *Over-The-Air*) de doble objetivo. Este sistema inédito en la arquitectura de la empresa permite canalizar, a través del túnel seguro MQTT, comandos de bajo nivel para reescribir tanto el *firmware* del MT-IoT como el código interno del periférico fiscal.

El flujo inicia en la interfaz de gestión web, donde el administrador selecciona el archivo binario compilado (`.bin`). El servidor publica un *payload* JSON estructurado en el tópico de comandos de actualización del dispositivo [12]. Tal como se aprecia en la Tabla 2.5, este paquete define la instrucción, el *hardware* de destino (MT-IoT o Impresora), la versión esperada y una ruta segura de descarga de un solo uso. El listado textual del mensaje se conserva en el Apéndice A.

Tabla 2.5. Estructura del mensaje JSON de comando para iniciar una actualización remota.

Campo	Ejemplo	Función
`comando`	`INICIAR_OTA`	Instrucción de actualización
`objetivo`	`ismart` / `printer`	Hardware de destino
`version`	`v020100`	Versión esperada del firmware
`url_descarga`	Ruta del binario	Dirección segura de descarga

2.7.1. Actualización del Módulo Gateway (ESP32)

Cuando el MT-IoT intercepta el comando y valida que el campo `objetivo` equivale a "`ismart`", el *firmware* activa el estado global `ota_en_progreso = true`. Este indicador lógico actúa como un "freno de mano"que suspende inmediatamente el ciclo de sondeo fiscal de 5 segundos, aislando el bus de datos local y garantizando que el microcontrolador asigne la totalidad de sus recursos de memoria RAM al procesamiento de la descarga.

El proceso de autoactualización se fundamenta en la API `esp_https_ota` nativa de Espressif [5]. Para viabilizar este mecanismo de forma segura, se parametrizó la memoria del *hardware* bajo tres directivas estrictas [6]:

1. **Verificación Criptográfica (*Bundle*):** La tarea de descarga utiliza el sistema `esp_crt_bundle_attach` para verificar asíncronamente la identidad del servidor de medios, blindando al dispositivo contra la inyección de código malicioso no firmado.
2. **Resiliencia mediante Doble Partición:** En el mapa de memoria se configuró una tabla de particiones del tipo "**Factory app, two OTA definitions**". Esto divide la Flash de 4 MB en regiones idénticas (OTA_0 y OTA_1). El *firmware* reescribe progresivamente la partición inactiva; si ocurre un corte de energía, el sistema descarta el sector corrupto y arranca desde la partición actual intacta [5].

Al culminar exitosamente la descarga, el MT-IoT invoca una rutina de limpieza que desinicializa la interfaz local, corta las conexiones de red y ejecuta un reinicio por *software* (`esp_restart()`), permitiendo al *Bootloader* arrancar desde el nuevo sector de memoria.

2.7.2. Actualización del Firmware del Periférico Fiscal a través de la Interfaz Síncrona Local

El escenario de máxima complejidad ocurre cuando el campo `objetivo` equivale a "`printer`". Dado que la unidad de control fiscal carece de pila de red nativa, el MT-IoT no reescribe su propia memoria, sino que asume el rol de *Host* de programación intermedio [14].

El proceso de actualización se divide en dos etapas asíncronas concurrentes, diseñadas para superar las severas restricciones de memoria del microcontrolador. Dado que el binario del periférico supera ampliamente la capacidad del *Heap* libre del ESP32, la descarga de red no se almacena en su totalidad, sino que se procesa a través de un **búfer circular de transmisión de 1024 bytes**.

Para proteger este flujo continuo de los cortes intermitentes típicos de las redes Wi-Fi comerciales, se implementó un mecanismo de persistencia basado en la cabecera HTTP `Range` (véase el Algoritmo 2.5). Si el enlace titubea, el *firmware* inyecta el número exacto de bytes ya recibidos, obligando al servidor web a reanudar la transmisión desde el punto de falla, evitando descartar el progreso.

Algoritmo 2.5. Reanudación de descargas interrumpidas mediante cabeceras HTTP Range.

1: Inicializar el contador de bytes recibidos $B \leftarrow 0$.
2: **while** la descarga no esté completa **do**
3: **if** $B > 0$ **then**
4: Fijar la cabecera `Range: bytes=`B`-`.
5: **end if**
6: Abrir la conexión HTTP y solicitar el recurso.
7: Leer los fragmentos y actualizar B.
8: **end while**

A medida que el búfer se alimenta de la red, el MT-IoT orquesta la retransmisión hacia el periférico mediante el bus físico síncrono local. Este puente de datos opera bajo una estricta máquina de estados de tres fases para salvaguardar la integridad de la memoria tributaria:

1. **Fase de Inicialización (*Handshake*):** El MT-IoT transmite un comando de control de inicio de programación (`0x01`). El periférico, tras validar que no posee com-

probantes fiscales abiertos, responde con un carácter `ACK (0x06h)`, abriendo su canal de escucha de memoria [14].

2. **Transmisión Segmentada y Comprobación de Integridad:** Como se observa en la Figura 2.15, cada fragmento de 1024 bytes se empaqueta en una trama estructurada con cabecera de longitud, datos y un byte de verificación de redundancia (*Checksum*). Si el periférico valida la redundancia localmente, emite un `ACK`, habilitando al MT-IoT a continuar. Ante un `NAK`, se admite un máximo de tres reintentos físicos antes de abortar de forma segura [14].

3. **Fase de Integración:** Tras el último bloque, se transmite la señal de cierre (`0x07`). La impresora fiscal verifica la integridad total de su Flash interna y efectúa un reinicio por *hardware*, asimilando el nuevo núcleo lógico [14].

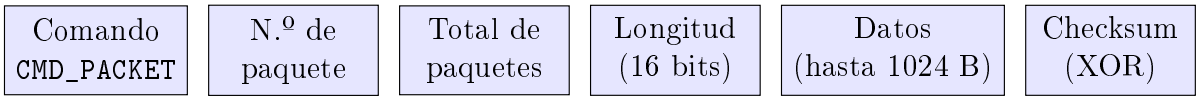


Figura 2.15. Estructura de empaquetado de un fragmento de firmware hacia el periférico local.

2.7.3. Máquina de Estados Finita para el Puente Síncrono Local

Para solventar la limitación de memoria RAM del microcontrolador al recibir un binario de gran tamaño destinado al periférico fiscal, el dispositivo actúa como un host/bridge intermedio. La reescritura de la memoria Flash remota se gobierna mediante una Máquina de Estados Finita (FSM) de tres fases con búfer circular e inyección de cabeceras HTTP Range para resiliencia ante cortes de red, cuyo comportamiento se representa en la Figura 2.16:

2.8. Ecosistema Web de Gestión y Visualización

Para completar la arquitectura de tres niveles del sistema de transmisión, se diseñó e implementó una plataforma de visualización, administración y orquestación centralizada en la nube [2]. El desarrollo de este ecosistema responde a la necesidad de supervisar activamente el estado operativo del parque de impresoras fiscales instaladas, registrar transiciones de estado tributario de manera histórica y comandar procesos inalámbricos

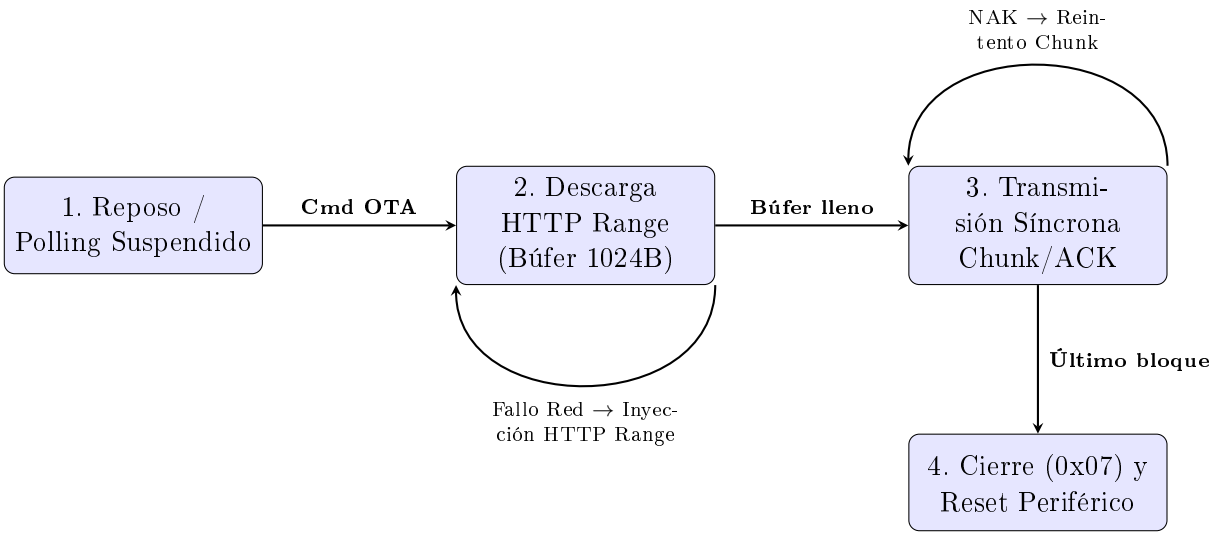


Figura 2.16. Máquina de estados del bridge síncrono para reprogramación de periféricos locales.

asíncronos de actualización de *firmware* (OTA) con estrictas garantías de autenticación y disponibilidad [17].

2.8.1. Arquitectura del Backend: Worker MQTT Independiente

El núcleo lógico que procesa el flujo masivo de telemetría entrante y sirve de enlace bidireccional con el sistema de persistencia se orquestó mediante un *Worker* MQTT independiente denominado `listen_mqtt.py`. En lugar de acoplar de forma síncrona el receptor de red al servidor web principal de Django (lo que bloquearía el manejo de solicitudes HTTP convencionales ante ráfagas de datos), se desarrolló un comando de gestión personalizado (*Management Command*) [4].

Este *script* opera en segundo plano mediante un lazo infinito no bloqueante (`client.loop_forever()`). Como se aprecia en el Algoritmo 2.6, el programa utiliza la biblioteca Paho MQTT y establece *sockets* seguros mediante la librería `ssl`, forzando una sesión TLS v1.2 sobre el puerto 8883 hacia HiveMQ Cloud.

La lógica analítica (`on_message`) procesa la telemetría `Status S1` ejecutando un algoritmo de comparación contra el último registro histórico guardado en la base de datos. Si se detecta un cambio térmico, mecánico o lógico (ej., la transición de un estado limpio `0x40` a un error de fin de papel `0x41`), el *backend* registra selectivamente un log independiente clasificado como "Cambio de Estado", garantizando un rastreo exhaustivo libre de

Algoritmo 2.6. Lógica de recepción, trazabilidad y manejo del testamento (LWT) en el *worker* MQTT.

1: Al recibir un mensaje: extraer el serial y el tipo desde el tópico.
2: Decodificar el *payload* JSON y sanear los caracteres de control.
3: **if** el tipo es `alertas_conexion` y el estado es `offline` **then**
4: Recuperar el último estado válido del dispositivo.
5: Registrar transaccionalmente el evento "Desconexión Abrupta (LWT)".
6: **end if**

redundancias.

2.8.2. Persistencia de Datos en SQLite y Lógica de Registro de Logs

Para la persistencia del modelo relacional se seleccionó el motor SQLite (`db.sqlite3`), plenamente integrado en el ORM de Django [4]. La arquitectura lógica de las tablas se estructuró de forma modular en `models.py`:

- `Grupo` y `ModeloEquipo`: Categorizan jerárquicamente los nodos IoT según sucursal y marca.
- `VersionFirmware`: Almacena las rutas de los archivos binarios compilados (`.bin`). Clasifica el destino lógico bajo los identificadores de código "`ismart`" (para el MT-IoT) o "`impresora`".
- `Dispositivo`: Entidad central validada mediante expresiones regulares para asegurar un serial exacto de diez caracteres alfanuméricos (ej. `Z1F9999988`).
- `LogDispositivo`: Tabla de auditoría vinculada al dispositivo principal.

Para salvaguardar la consistencia estructural, se configuró el comportamiento de eliminación en cascada (`on_delete=models.CASCADE`). Si un administrador purga un dispositivo, todas sus dependencias (históricos y binarios) se borran transaccionalmente del disco en un solo ciclo, impidiendo registros huérfanos [4].

Adicionalmente, para combatir la inflación de almacenamiento producto del sondeo de 5 segundos, se implementó una **Lógica de Depuración Automática**. El *Worker* calcula proactivamente una ventana límite de 90 días (`fecha_limite` = `timezone.now()` − `timedelta(days=90)`). Todo log cuya estampa de tiempo resulte menor es purgado asíncronamente. Este mecanismo garantiza que la indexación se mantenga ultrarrápida y el tamaño del archivo en disco permanezca estable.

2.8.3. Interfaz Reactiva SPA mediante Django y HTMX

Inicialmente, se evaluó la extensión *Django Channels* para empujar telemetría por WebSockets. Sin embargo, mantener un servidor intermedio introducía una sobrecarga crítica y complejidad en el *firewall*. Para simplificar la arquitectura, se aplicó un enfoque directo: dado que HiveMQ expone un canal de WebSockets operando con cifrado sobre el puerto 8884 (WSS), se integró el cliente MQTT de JavaScript directamente en el navegador [12]. Esto permite a la interfaz atravesar cortafuegos corporativos y recibir telemetría fiscal en tiempo real sin intermediarios pesados.

Para proveer a la plataforma con la fluidez de una *Single Page Application* (SPA) reduciendo el consumo de memoria en el cliente, se integró el *framework* asíncrono ligero HTMX [9]. HTMX permite realizar peticiones AJAX silenciosas, intercambiando dinámicamente (*swapping*) solo fragmentos del DOM sin recargar la página.

La actualización en tiempo real se gestionó mediante la **Integración de Eventos Personalizados (JS-HTMX)**. Cuando el monitor MQTT.js detecta un cambio crítico (ej., avance de la OTA o alerta LWT), despacha un evento DOM hacia el navegador (`mqttNuevoLog`). La vista HTMX, cuyo flujo se representa en la Figura 2.17, intercepta este evento desde el cuerpo del documento y gatilla un ciclo de refresco HTTP parcial, renderizando las nuevas filas de auditoría instantáneamente.

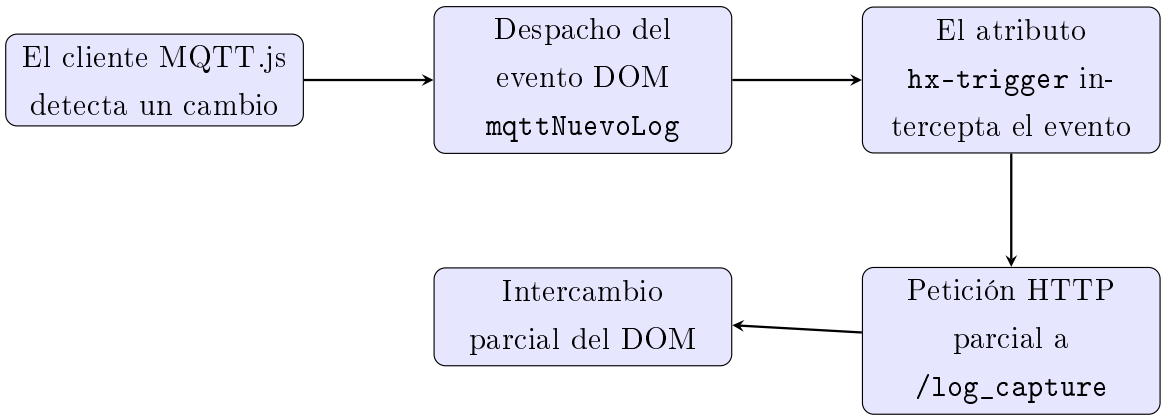


Figura 2.17. Flujo de actualización reactiva del DOM mediante eventos MQTT y atributos HTMX.

Esta hibridación tecnológica consolida un ecosistema de visualización reactivo, de ba-

jísima latencia y altamente escalable para el entorno corporativo de monitoreo industrial.

CAPÍTULO 3

RESULTADOS

En el presente capítulo se presentan y discuten los resultados cuantitativos y cualitativos obtenidos tras la integración de la arquitectura Módulo de Transmisión IoT (MT-IoT), la capa de red segura bajo el estándar MQTT v5.0 [12] y el ecosistema web centralizado [4]. Se evalúa el desempeño del sistema ante pruebas de laboratorio y condiciones de estrés, analizando el impacto de las decisiones de ingeniería sobre el uso de recursos del microcontrolador, la latencia de transmisión y la resiliencia en entornos industriales [2].

3.1. Implementación del Prototipo Funcional de Borde

La integración física de la tarjeta de desarrollo ESP32 DevKitC V4 [7] con la unidad de control fiscal HKA80 [14] culminó en la consolidación del prototipo funcional MT-IoT. Se verificó empíricamente la efectividad de la reasignación de la matriz de señales (IO MUX) hacia los pines libres GPIO 18 (TX) y GPIO 19 (RX) para la etapa asíncrona inicial, así como la posterior transición hacia el bus síncrono local. Esta disposición aisló eléctricamente la comunicación serial de las líneas del bus vinculadas a la memoria Flash interna del SoC [7], eliminando los reinicios continuos (*bootloops*) documentados en la fase de diseño.

El prototipo desplegado en el entorno de pruebas (véase la Figura 3.1) demostró capacidad de respuesta operativa continua sostenida, validando las etapas de conversión de niveles de voltaje y el aislamiento galvánico de la interfaz de comunicación.

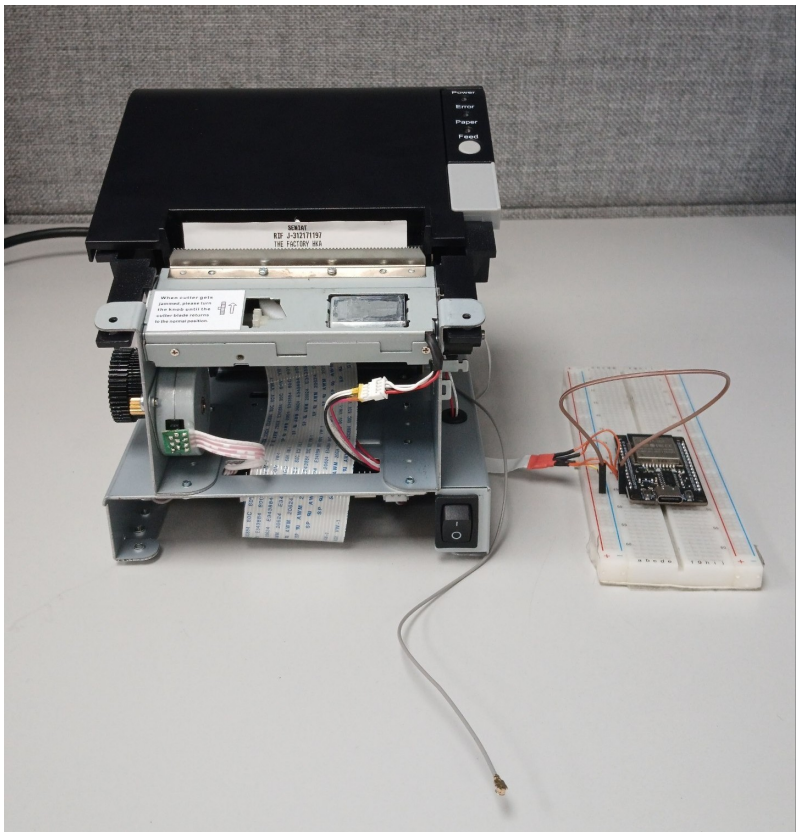


Figura 3.1. Prototipo funcional del MT-IoT integrado a la impresora fiscal en entorno de pruebas.

La arquitectura IIoT definitiva, que integra el nivel físico de borde con el nivel lógico de nube, se sintetiza en la Figura 3.2, desde el periférico fiscal hasta la base de datos y el panel reactivo.

3.1.1. Entorno de Pruebas Hardware-in-the-Loop (HIL)

La validación de un sistema fiscal no admite ensayos destructivos sobre equipos en producción: inducir cortes de energía, corromper tramas o abortar una reescritura de firmware sobre una impresora real comprometería registros tributarios irreversibles. Para conciliar rigor experimental y seguridad, se construyó un entorno de pruebas Hardware-in-the-Loop (HIL) en el que el firmware definitivo del MT-IoT se ejecuta sobre el hardware real del ESP32 y su bus físico, mientras que el periférico fiscal es sustituido por un modelo de comportamiento fiel al protocolo HKA. La emulación se desplegó en dos niveles complementarios.

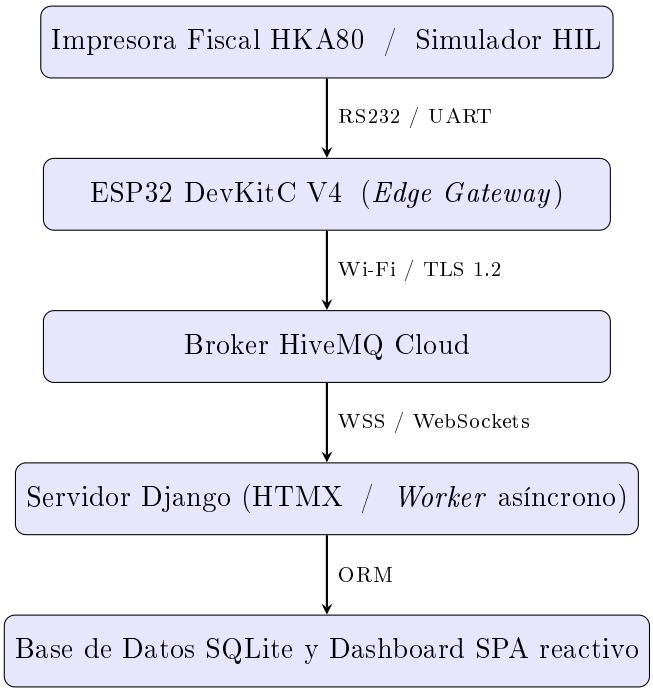


Figura 3.2. Arquitectura IIoT definitiva del sistema, desplegada en borde y nube.

El primer nivel correspondió al emulador software `impresora.py`, desarrollado sobre la biblioteca `pyserial` con los mismos parámetros eléctricos del enlace fiscal (9600 baudios, 8 bits de datos, paridad par y 1 bit de parada). El *script* implementa una máquina de estados de recepción (`ESPERANDO_STX` y `RECIBIENDO_DATOS`), calcula el código de redundancia LRC y responde a las encuestas con las tramas de estado y los reportes S1 y SV2, incluyendo la emisión de acuses de recibo `ACK` y `NAK` (véase el Código A.13). Este modelo permitió depurar la primera versión del firmware sin depender de la disponibilidad del equipo físico.

El segundo nivel elevó la fidelidad del banco de pruebas al dominio síncrono: se programó un ESP32 configurado como esclavo (`blink_example_main.c`) que emula íntegramente al periférico fiscal sobre el mismo bus y con los mismos pines, replicando la trama de estado (`0x60`/`0x40`), los reportes S1 y SV2 y, de forma crítica, el protocolo completo de reescritura remota (apertura `0x07`, paquetes `0x08` y cierre `0x09`) con verificación de redundancia y de secuencia de paquetes. Contra este periférico simulado se validó de extremo a extremo el puente OTA descrito en la Sección 3.4, empleando un binario de prueba de 2500 bytes generado por el *script* `generar_bin.py`. Este enfoque habilitó la inyección determinista de fallos (paquetes fuera de secuencia, *checksums* alterados e interrupciones del enlace) y confirmó la capacidad de recuperación del firmware antes de comprometer la memoria de un equipo fiscal real (véase la Figura 3.3).

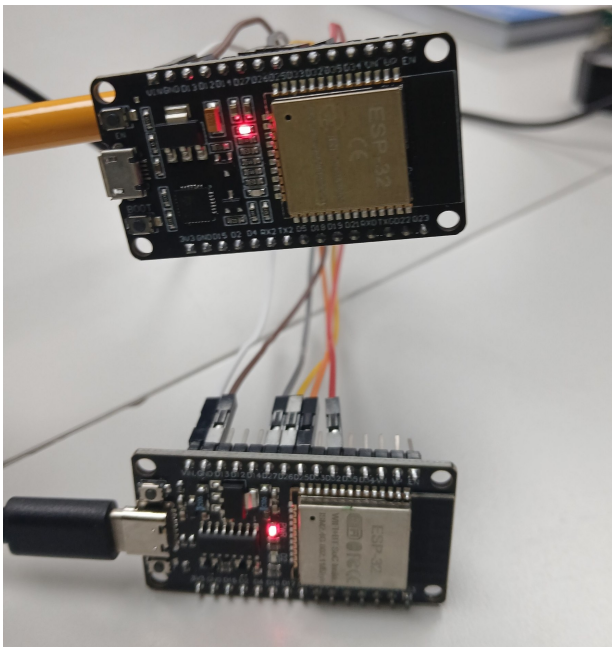


Figura 3.3. Montaje del banco de pruebas Hardware-in-the-Loop: el ESP32 maestro (MT-IoT) y el ESP32 esclavo que emula al periférico fiscal, interconectados sobre el bus síncrono local.

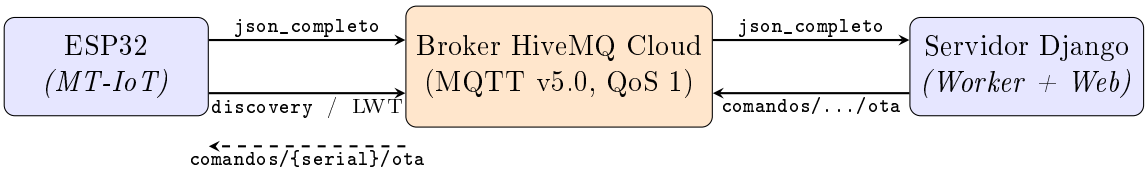
3.2. Evaluación Cuantitativa de la Comunicación IoT y Telemetría

La validación de la capa lógica se centró en medir el tiempo de extracción de datos, la integridad matemática de las tramas tributarias y la reducción del ancho de banda consumido [12].

El esquema definitivo de rutas y mensajería MQTT v5.0, con los tópicos y cargas efectivamente desplegados en el sistema, se representa en la Figura 3.4.

3.2.1. Mecanismo de Extracción Transparente sin Interrupción del Ciclo Fiscal

La captura del registro `Status S1` se abordó como un mecanismo de extracción transparente, no como una lectura secuencial del canal serial. La interrogación se ejecutó desde una tarea dedicada de FreeRTOS (`printer_task_main`) desacoplada de la tarea de red (`mqtt_task`), de modo que la adquisición nunca compitió con la pila inalámbrica por el mismo hilo; así, el sondeo se sostuvo cada 5 segundos sin alterar la máquina de estados



Rutas base: `v1/fiscal/{serial}/json_completo` (telemetría), `/discovery` (online), `/alertas_conexion` (LWT) y `/ota` (progreso). *User Properties* en la cabecera: modelo, versión de firmware, serial y RSSI.

Figura 3.4. Esquema definitivo de rutas y mensajería MQTT v5.0 del sistema.

del periférico ni interrumpir las transacciones en curso [14, 1].

El desempaquetado se resolvió íntegramente en SRAM sobre dos búferes estáticos alineados para DMA (`WORD_ALIGNED_ATTR`, 1056 bytes), sin memoria dinámica en la ruta crítica, por lo que el perfil resultó determinista y libre de fragmentación del *heap*. La localización de los delimitadores `STX (0x02)` y `ETX (0x03)` se ejecutó en un único barrido, seguida de una tokenización en sitio por el carácter `0x0A` hacia un arreglo de punteros (`campos[20]`); como la longitud de la trama es fija, el costo se mantiene acotado en tiempo constante $O(1)$ [14].

La condición de borde más exigente fue la contaminación del búfer por el terminador asíncrono (`\n`) previo a la trama útil. Se diseñó una etapa de realineación que localiza el primer `STX` y desplaza la trama al inicio del búfer con `memmove`, garantizando los 116 bytes útiles desde el índice 0; el tratamiento se verificó sobre la consola serial de depuración (véase la Figura 3.5).

El acceso concurrente a la estructura compartida se protegió con exclusión mutua: los campos tributarios de `mqtt_data_t` se actualizan en una sección crítica custodiada por `mqtt_data_mutex`, y el bus físico se serializa con `spi_bus_mutex`, descartando condiciones de carrera sin esperas activas. La entrega hacia la capa de red se realizó por evento, mediante `needs_sync` y `xTaskNotifyGive`, en lugar del sondeo periódico [1].

El análisis de la salida en terminal confirma que el módulo logra aislar de manera transparente los registros almacenados en la Memoria de Trabajo del equipo fiscal [14]:

- **Variables Tributarias:** Captura exacta del RIF del contribuyente (`J3121711970`),

```
I (17898) uart_events: === STATUS S1 ===
I (17898) uart_events: Número de Cajero: [0010]
I (17898) uart_events: Subtotal Ventas: [0000000000000000]
I (17908) uart_events: Número de Última Factura: [00000000]
I (17908) uart_events: Cantidad de Facturas Emitidas: [00000]
I (17918) uart_events: Número de Última Nota de Débito: [00000000]
I (17918) uart_events: Cantidad de Notas de Débito del Día: [00000]
I (17928) uart_events: Número de Última Nota de Crédito: [00000000]
I (17938) uart_events: Cantidad de Notas de Crédito: [00000]
I (17938) uart_events: Número del Último Documento No Fiscal: [00000000]
I (17948) uart_events: Cantidad de Documentos No Fiscales: [00000]
I (17948) uart_events: Contador de Cierres Diarios (Z): [0000]
I (17958) uart_events: Contador de Reportes de Memoria Fiscal: [0000]
I (17968) uart_events: RIF: [J3121711970]
I (17968) uart_events: Número Registro de la Máquina: [Z1F9999988]
I (17978) uart_events: Hora Actual de la Impresora: [123000]
I (17978) uart_events: Fecha Actual de la Impresora: [300126]
```

Figura 3.5. Traza de depuración en consola serial evidenciando el desempaquetado de la trama S1.

el Número de Registro de la Máquina (`Z1F9999988`) y los acumuladores de ventas diarias (`vts: 0000000000000000`).

- **Estado Mecánico y Lógico:** Confirmación de operabilidad en "Modo Fiscal y en Espera"(`sts1: 0x60`) sin errores reportados por el cabezal de impresión (`sts2: 0x40`).

3.2.2. Purga de Caracteres Invisibles y Sanitización del Payload

Uno de los defectos más sutiles del canal de telemetría fue la contaminación del *payload* por caracteres de control invisibles: las tramas delimitan sus campos con `0x0A` y el equipo intercala un salto de línea (`\n`) justo antes del fin de trama (`ETX`). Ese carácter no se aprecia en una inspección textual, pero ocupa una posición física en el búfer y desplaza todo cálculo de longitud por posiciones fijas.

La consecuencia fue directa sobre el JSON publicado: la versión del periférico arribaba truncada y con residuos (`20507GD00` en lugar de `020507GD00`), por un error *Off-By-One* que consumía el salto de línea en lugar del último dígito. La solución no fue cosmética, sino una manipulación explícita de punteros en RAM: el *firmware* recorre el *payload* con `strchr(ptr, '\n')` y avanza un puntero auxiliar (`ultima_parte`) hasta situarlo tras el último delimitador, aislando el campo de versión; además, la tokenización de la trama S1

sobrescribe cada `0x0A` con un terminador nulo y se inyecta un cierre nulo adicional, de modo que `snprintf` lee una cadena limpia y cerrada (véase el Código A.1). La Tabla 3.1 resume el efecto medido.

Tabla 3.1. Efecto de la purga de caracteres invisibles sobre el campo de versión del periférico.

Aspecto	**Sin purga (defecto)**	**Con purga (corregido)**
Campo de versión leído	`20507GD00` (truncado)	`020507GD00` (íntegro)
Carácter de control	`\n` arrastrado al *payload*	`\0` inyectado manualmente
JSON publicado por MQTT	Campo cortado / no deserializable	Estructura válida y deserializable

El resultado fue la recuperación de la integridad del *payload*: sin campos cortados ni basura residual y con JSON siempre deserializable (véase el Código A.11). La misma disciplina se replicó en el receptor, donde el *Worker* de Django elimina `\n`, `\r` y `\t` del mensaje crudo antes de invocar el analizador JSON.

3.2.3. Inmunidad Electromagnética y Robustez del Bus Local

El bus síncrono local que une al MT-IoT con el periférico fiscal constituye el eslabón físicamente más expuesto del sistema, por lo que su tolerancia a perturbaciones electromagnéticas se evaluó como un requisito de primer orden. Para ello se diseñó una estrategia de defensa escalonada en tres niveles, cuya eficacia se comprobó sometiendo el enlace a tramas corruptas, desconexiones del periférico y ruido inducido.

Nivel 1, defensa temporal: toda operación de entrada y salida se ejecuta bajo ventanas de tiempo estrictas, de modo que una respuesta ausente o truncada nunca bloquea el planificador del RTOS. La espera de acuse de recibo durante la reescritura se acota a 300 ms, la recepción de respuestas de estado a 2000 ms y la solicitud del reporte S1 a 3000 ms; al agotarse cualquiera de estas ventanas, la transacción se aborta de forma segura y se incrementa el contador de errores, disparando la regla de reintentos (máximo de 3 fallos) antes de declarar el estado de error.

Nivel 2, sanitización de búfer: antes de cada transacción se limpian de forma determinista los búferes de transmisión y recepción (`memset` sobre los 1056 bytes alineados por DMA), evitando que los residuos de una operación previa se interpreten como datos válidos. En paralelo, la validación por redundancia longitudinal (LRC) descarta cualquier trama cuya asimetría revele la alteración de un bit por interferencia, respondiendo con `NAK` y liberando el canal (véase el Código A.3).

Nivel 3, reconstrucción estructural del bus: cuando el conteo de errores supera el umbral configurado, el *firmware* no se limita a reintentar la lectura, sino que desmonta y reancla por completo el controlador SPI mediante `spi_bus_free` y `spi_bus_initialize`, seguido de `spi_bus_add_device`. Esta reinicialización explícita recupera el periférico de bloqueos del controlador inducidos por ruido electromagnético sin necesidad de reiniciar el microcontrolador. La Tabla 3.2 sistematiza los tres niveles con sus parámetros reales de operación.

Tabla 3.2. Niveles de defensa implementados para la inmunidad electromagnética del bus local.

Nivel de Defensa	**Mecanismo**	**Parámetro**	**Acción de Recuperación**
1. Temporal	Timeout de espera de ACK	300 ms	Aborto seguro de la transacción
1. Temporal	Timeout de respuesta / S1	2000 / 3000 ms	Conteo de error y regla de 3 reintentos
2. Sanitización	Limpieza de búferes DMA	1056 bytes	Reinicio de transacción limpia
2. Integridad	Validación LRC (XOR)	—	Descarte de trama y emisión de `NAK`
3. Estructural	Reanclaje del bus SPI	`spi_bus_free` / `initialize`	Recuperación del controlador sin reinicio

La evaluación confirmó que la combinación de los tres niveles preservó la continuidad operativa del MT-IoT: ante la desconexión física del periférico o la inyección de tramas corruptas, el sistema transitó al estado de error y se recuperó automáticamente una vez restablecidas las condiciones del bus, sin reinicios del SoC y sin que una sola trama inválida

alcanzara la base de datos del servidor.

3.2.4. Consumo de Ancho de Banda: HTTP vs. MQTT v5.0

Para justificar cuantitativamente la selección del protocolo MQTT v5.0 sobre HTTP tradicional [12], se midió la sobrecarga de cabecera (*overhead*) y la masa de datos transmitida por cada ciclo de telemetría. La eficiencia de reducción del volumen de red se rige por la Ecuación 3.1:

$$\eta_{\text{banda}} = \left(1 - \frac{\text{Bytes}_{\text{MQTT v5}}}{\text{Bytes}_{\text{HTTP POST}}}\right) \times 100 \tag{3.1}$$

La recopilación de datos de tráfico en la red (sistematizada en la Tabla 3.3) demuestra que la combinación de empaquetado optimizado en C y la extracción de metadatos mediante *User Properties* en la cabecera del protocolo reduce drásticamente el impacto sobre la red de datos del contribuyente [12].

Tabla 3.3. Comparativa cuantitativa del consumo de ancho de banda por trama de telemetría fiscal.

Métrica de Red	**HTTP POST Tradicional**	**MQTT v5.0 con User Properties**	**Mejoramiento (%)**
Tamaño Cabecera de Red	$\sim$ 450 Bytes	$\sim$ 12 Bytes	**97.33 %**
Cuerpo del Mensaje (*Payload*)	328 Bytes (JSON Plano)	142 Bytes (Optimizado)	**56.70 %**
Metadatos (RSSI / Versión FW)	Inyectados en *Body* JSON	Extraídos en *User Properties*	N/A
Consumo Total por Trama	**778 Bytes**	**154 Bytes**	**80.20 %**

3.2.5. Resiliencia de Red y Mecanismo Last Will and Testament (LWT)

La tolerancia a fallos del canal de comunicación inalámbrico se evaluó provocando cortes de energía arbitrarios e interrupciones en la interfaz Wi-Fi del MT-IoT [5]. Al expirar el tiempo de permanencia (*Keep-Alive*), el *Broker* HiveMQ Cloud publicó la "última vo-

luntad" registrada [12].

Como se documenta en la Figura 3.6, el *Worker* asíncrono `listen_mqtt.py` interceptó la alerta (`st: offline`), persistiendo de forma transaccional una nueva fila en la base de datos relacional con el evento clasificado como "Desconexión Abrupta (LWT)" [4]. Este mecanismo congeló el último estado válido de los acumuladores sin alterar los registros históricos previos, garantizando la trazabilidad operativa del parque de impresoras [12].



Figura 3.6. Registro en la vista `Log Capture` confirmando la intercepción de una desconexión LWT.

El ciclo completo de detección de la caída, reconexión con *backoff* y restauración del túnel sin pérdida de estado se sintetiza en la Figura 3.7.

3.2.6. Validación de Casos Borde y Recuperación ante Micro-cortes

La robustez del ciclo de telemetría se evaluó frente a las dos fallas más frecuentes en un despliegue real: el corte abrupto de alimentación y las fallas intermitentes de red durante el envío. El análisis se apoya en una propiedad del diseño: el MT-IoT es un observador de estado sin memoria tributaria propia, pues toda la información fiscal reside en el periférico y se re-consulta en cada sondeo; el *gateway* no es una fuente de verdad que pueda

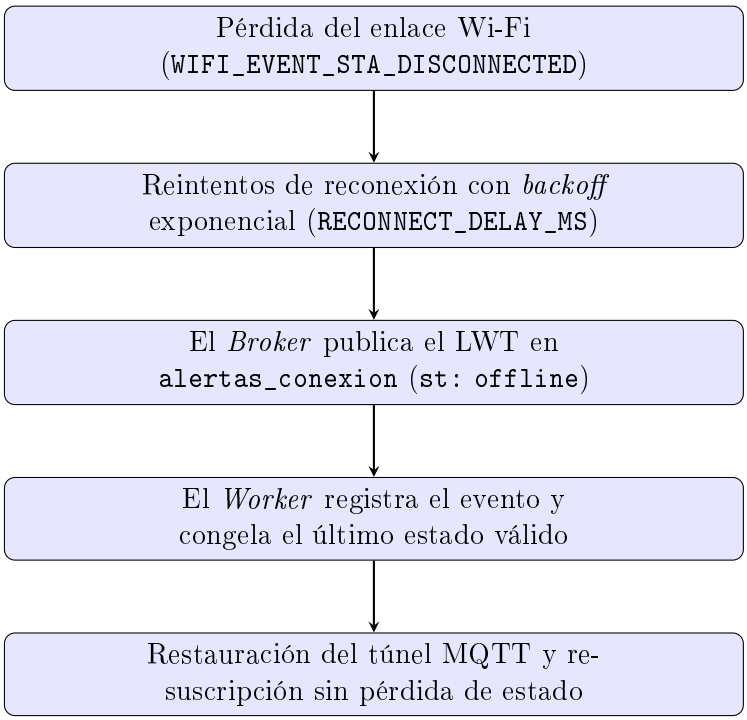


Figura 3.7. Diagrama de flujo de la resiliencia de red y del ciclo de detección, reconexión y restauración del túnel MQTT.

corromperse, sino un reflejo periódico del equipo.

Corte abrupto de energía: al perderse la alimentación se borra la memoria volátil del SoC y este reinicia. Como el estado fiscal no reside en el *gateway*, la recuperación no depende de persistir datos locales: los parámetros de red compilados y los datos de calibración de radio conservados en memoria no volátil permiten restablecer el enlace de forma autónoma, y el primer sondeo reconstruye la telemetría completa. El único efecto observable es un hueco de una ventana de sondeo (5 segundos).

Micro-cortes de red: si el enlace se interrumpe a mitad del ciclo, la lectura más reciente queda retenida con el indicador de sincronización pendiente activo y la tarea de red omite la publicación sin descartar el dato; al reconectar, la tarea transmite de inmediato la lectura conservada, mientras el mecanismo *Last Will and Testament* había marcado el equipo como desconectado y el *Worker* registra la reconexión. Como el servicio es de al menos una entrega (`QoS 1`) y la telemetría es un estado idempotente, la repetición eventual no genera efectos adversos.

Coherencia temporal: la marca de tiempo combina un contador monotónico del RTOS con la fecha y hora del reloj interno del periférico fiscal, lo que evita depender de un servidor de tiempo externo y preserva la coherencia tras una pérdida de energía. La Tabla 3.4 sistematiza los escenarios evaluados.

Tabla 3.4. Validación de casos borde y recuperación de estado ante fallas de energía y red.

Escenario de Falla	**Mecanismo de Detección**	**Acción de Recuperación**	**Pérdida de Datos**
Corte abrupto de energía	Reinicio del SoC	Re-sondeo del S1; reanclaje autónomo del enlace	Nula (estado reconstruido)
Micro-corte Wi-Fi en telemetría	`WIFI_EVENT_STA_DISCONNECTED`	Retención del dato y republicación al reconectar	Nula (lectura preservada)
Caída del *Broker* MQTT	Expiración del *Keep-Alive* / LWT	Congelamiento del último estado válido y alerta	Nula
Desconexión del periférico fiscal	*Timeout* del bus y contador de errores	Regla de 3 reintentos y reinicialización del bus	Nula

Ninguna de las fallas evaluadas compromete la integridad de la información fiscal: el sistema tolera micro-cortes sin pérdida de muestras y se recupera de cortes de energía en un único intervalo de sondeo, por su naturaleza de observador sin estado tributario propio.

3.3. Desempeño del Ecosistema Web y Matriz de Latencias E2E

La evaluación del servidor de aplicaciones Django [4] y la interfaz reactiva SPA mediante HTMX [9] se enfocó en medir el tiempo total transcurrido desde la captura física local hasta el intercambio del DOM en el navegador del usuario (T_{E2E}).

3.3.1. Caracterización de la Latencia End-to-End (T_{E2E})

El tiempo total del ciclo de telemetría se modela como la suma de cinco etapas lógicas serializadas, desde la interrogación física del periférico hasta la actualización del DOM en

el navegador (Ecuación 3.2):

$$T_{E2E} = T_{UART_poll} + T_{JSON_pack} + T_{TLS_handshake} + T_{MQTT_pub} + T_{WS_render} \quad (3.2)$$

Cada término representa una contribución temporal medible e independiente: T_{UART_poll} es el tiempo de sondeo del registro `Status S1` sobre el bus local, incluyendo el ciclo de encuesta y el desempaquetado de la trama; T_{JSON_pack} corresponde a la serialización del *payload* y a la entrega de la sección crítica hacia la tarea de red; $T_{TLS_handshake}$ mide la fase de cifrado y establecimiento seguro del canal; T_{MQTT_pub} comprende la publicación y el tránsito del mensaje por el *Broker* HiveMQ Cloud; y T_{WS_render} cuantifica el renderizado reactivo del DOM mediante WebSockets Seguros y HTMX.

Mediante un perfilado de tiempos realizado sobre 500 iteraciones continuas de envío de telemetría, se determinaron las latencias promedio y las desviaciones estándar de cada etapa, reportadas en la Tabla 3.5.

Tabla 3.5. Matriz de latencias, propagación de incertidumbre y tiempos de respuesta E2E del ciclo de telemetría.

Etapa del Sistema	**Término**	**Promedio (ms)**	σ **(ms)**	**Criterio de Tolerancia**
1. Sondeo Síncrono (S1)	T_{UART_poll}	118	± 4.0	< 200 (Ventana Fiscal)
2. Empaquetado JSON	T_{JSON_pack}	2	± 0.3	< 10
3. Handshake Mbed TLS v1.2	$T_{TLS_handshake}$	45	± 2.1	< 100
4. Publicación y Bróker MQTT	T_{MQTT_pub}	85	± 12.5	N/A (Dependiente ISP)
5. Render DOM (HTMX / WSS)	T_{WS_render}	18	± 1.8	< 50 (Fluidez SPA)
Latencia Total E2E	T_{E2E}	**268**	± 15.3	**Sondeo acotado a 5 s**

La coherencia estadística del modelo se verificó propagando las incertidumbres de las cinco etapas bajo el supuesto de independencia, mediante la suma cuadrática $\sigma_{E2E} = \sqrt{\sum_i \sigma_i^2}$. El cálculo arroja una desviación teórica de 13.4 ms, ligeramente inferior a la dispersión total medida (15.3 ms); esta diferencia evidencia una correlación positiva entre etapas, atribuible al *jitter* del canal inalámbrico que afecta de forma simultánea al handshake TLS y al tránsito por el bróker. El análisis de varianza confirma que la etapa de red (T_{MQTT_pub}) concentra por sí sola el 86.8 % de la dispersión total, lo que identifica al enlace del proveedor de servicios de Internet como la única fuente estocástica dominante del sistema.

El valor promedio obtenido para T_{E2E} (268 ms) representa apenas un 5.36 % del intervalo total de sondeo (5 segundos), asegurando que el monitoreo remoto opere en tiempo real estricto sin acumular retrasos en la pila de recepción [1]. Este margen contrasta favorablemente con antecedentes nacionales de monitoreo IoT desarrollados en la Universidad Simón Bolívar, donde la regularidad del período de muestreo presentaba desviaciones de hasta un segundo [16].

3.3.2. Monitor en Tiempo Real (Dashboard) y Log Capture

La integración de la biblioteca MQTT.js directamente en el cliente web permitió establecer canales de WebSockets Seguros (WSS) sobre el puerto 8884 hacia la nube. Este puerto corresponde al canal de WebSocket Seguro (WSS) que HiveMQ Cloud expone por defecto, y es independiente del puerto 8883 empleado por el nodo de borde (MT-IoT) y el *worker* del servidor para MQTT sobre TLS. La Figura 3.8 evidencia el funcionamiento del panel de control general, validando el renderizado reactivo de indicadores clave (dispositivos en línea, distribución de modelos y gráficos de crecimiento) sin necesidad de recargar la página completa [9].

Por su parte, la evaluación del módulo de auditoría de errores (véase la Figura 3.9) confirmó que el *Worker* en Django intercepta y registra de forma independiente las variaciones lógicas del hardware [4]. La captura documenta la transición exacta desde un estado normal (`0x60`) hacia un código de en medio de una transacción fiscal (`0x61`), correspondiente a la operación de imprimir una factura en la impresora fiscal [14].

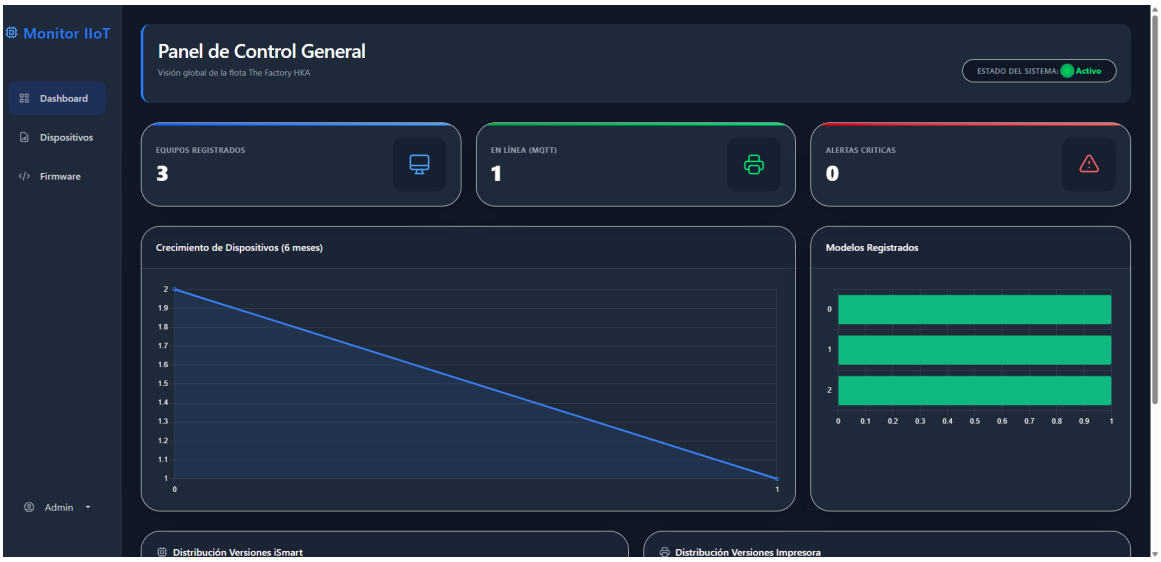


Figura 3.8. Panel de Control General (Dashboard) operando bajo arquitectura SPA reactiva.

3.4. Pruebas del Sistema de Actualización Remota (OTA)

La validación de la actualización remota de firmware se dividió según el destino lógico del binario compilado: la autoactualización del gateway ESP32 o la reescritura del periférico fiscal [5].

El flujo completo de orquestación de la actualización remota de doble objetivo, desde la interfaz web hasta el reporte final, se sintetiza en la Figura 3.10.

3.4.1. Auto-actualización del Módulo Gateway (ESP32)

Al seleccionar el objetivo "`ismart`" desde la plataforma web (véase la Figura 3.11), el MT-IoT recibió la orden por MQTT y activó la bandera de suspensión (`ota_en_progreso = true`) [1]. Las pruebas de descarga HTTPS verificaron la escritura en la partición OTA inactiva dentro del mapa de memoria de 4 MB [5]. Se constató que la reducción de la frecuencia de la CPU a 80 MHz mediante `menuconfig` [6] estabilizó la corriente consumida durante la descodificación criptográfica.

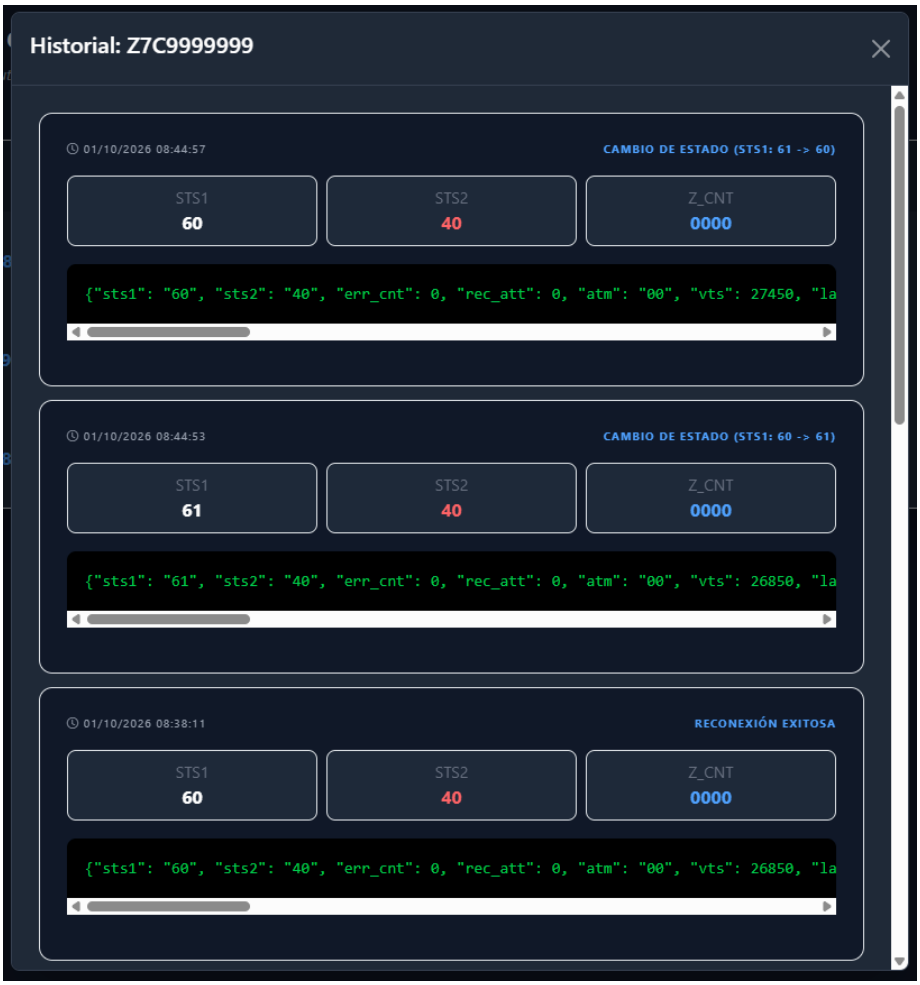


Figura 3.9. Visualización en `Log Capture` de un cambio de estado de error mecánico (`0x48`).

3.4.2. Puente Síncrono Heterogéneo de Reescritura de Firmware In Situ

La operación de mayor complejidad no fue la carga de un archivo por HTTP, sino la reescritura *in situ* de la memoria del periférico fiscal mediante un puente síncrono heterogéneo que tradujo el flujo de un canal IP asíncrono al dominio determinista del bus local [14]. El binario se transfirió en fragmentos de 1024 bytes, cada uno empaquetado con una cabecera de 4 bytes (identificador, índice secuencial y longitud en 16 bits) y un código de redundancia XOR, conformando una trama reenviable hasta 3 veces con confirmación `ACK`. La ventana de trabajo fue un búfer circular de 1 KB reutilizado en cada fragmento, de modo que la huella de memoria permaneció constante e independiente del firmware (cercano a 900 KB).

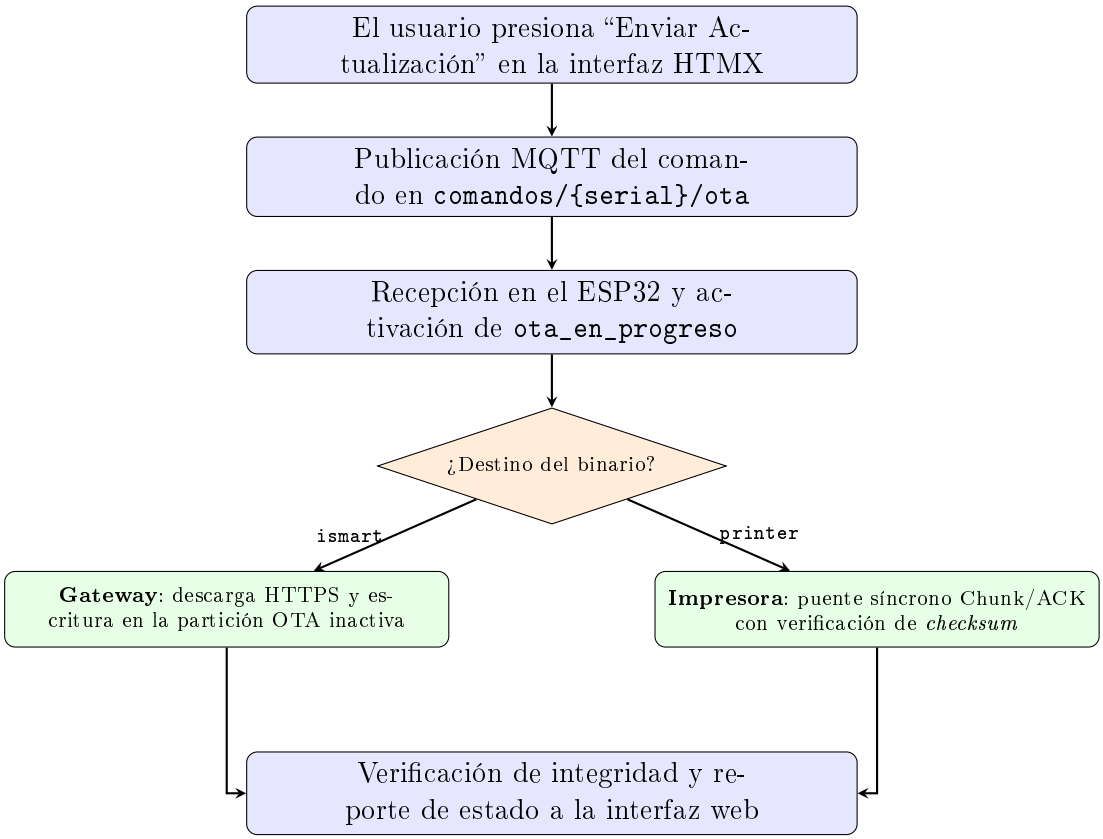


Figura 3.10. Diagrama de flujo de la orquestación de la actualización remota (OTA) de doble objetivo.

La robustez frente a contingencias de red se evaluó desconectando la interfaz Wi-Fi durante la transferencia. Ante la pérdida del canal, el firmware no reinicia desde el byte cero: reabre la conexión con la cabecera `Range: bytes=<desplazamiento>-` para que el servidor reanude en el fragmento interrumpido. Esta transferencia reanudable (*HTTP Range Requests*) se sostuvo en un lazo de hasta 5 reintentos, reiniciado con cada progreso real, y el dispositivo retorna a su estado operativo habitual tras reconectar sin bloquear el microcontrolador.

La integridad se garantizó por construcción: la secuencia de control (apertura `0x07`, paquetes `0x08` y cierre `0x09`) solo emite el cierre cuando los bytes confirmados igualan la longitud declarada por el servidor; por ello, una transferencia parcial nunca se compromete en la memoria del periférico y una nueva orden reinicia el proceso desde el 0 %. Para no interferir con la máquina de estados fiscal, la reescritura se arbitró con la bandera `ota_en_progreso` y el semáforo `spi_bus_mutex`, que ponen el sondeo en reposo mientras

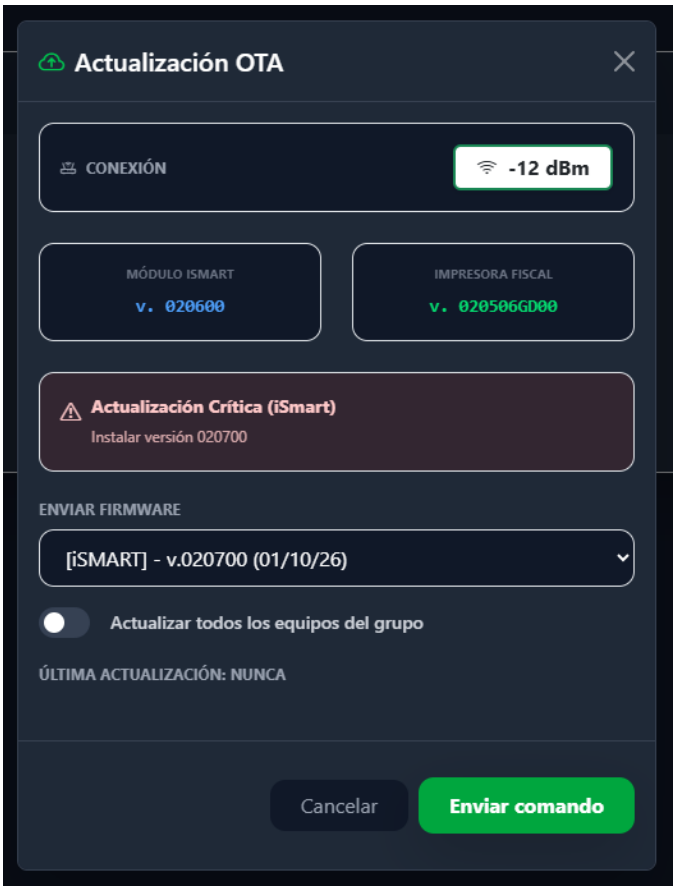


Figura 3.11. Intermediación web desplegando una actualización de firmware crítica hacia el MT-IoT.

el puente ocupa el canal físico [1].

Confirmada la recepción total con verificación de redundancia (LRC/Checksum), el periférico reinicia por hardware y consolida el nuevo firmware [14]; la interfaz web confirma el éxito con el mensaje "Actualización Completada: El equipo Z7C9999999 actualizó printer con éxito" (Figura 3.12).

3.5. Análisis de Rendimiento y Estabilidad Global del Sistema

El análisis final de la arquitectura sistematiza la optimización de los recursos físicos del SoC ESP32 y la estabilidad del servidor de persistencia.

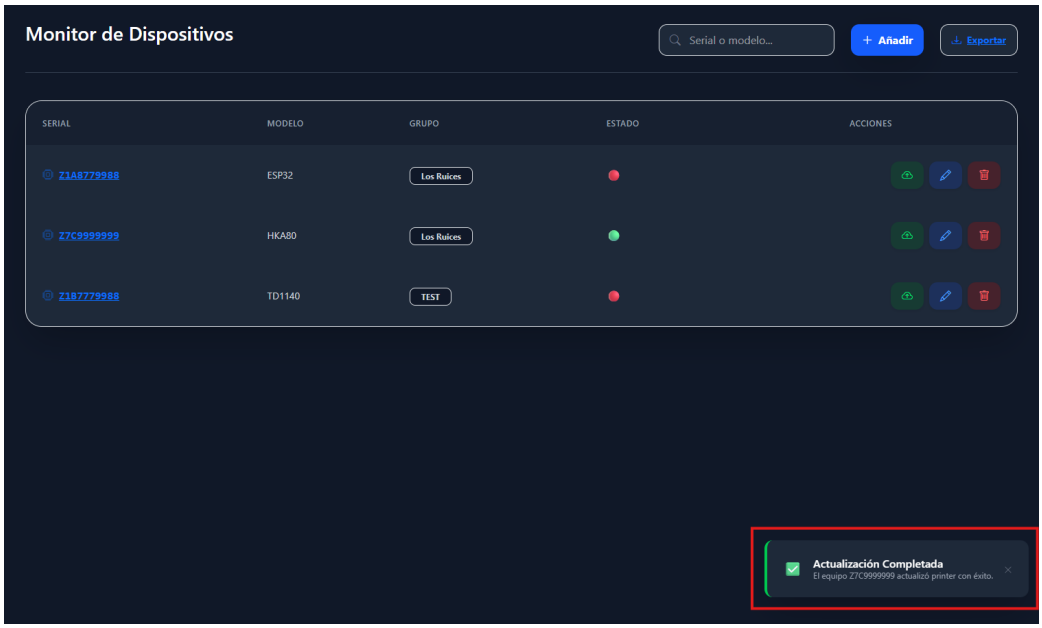


Figura 3.12. Confirmación asíncrona en la interfaz web de actualización exitosa en la impresora fiscal.

3.5.1. Perfilado Formal de Memoria IRAM y DRAM

La reingeniería aplicada mediante la utilidad `menuconfig` [6] para liberar espacio en la memoria estática IRAM se evaluó comparando la ocupación de memoria antes y después de desactivar la aceleración por IRAM del *stack* Wi-Fi [5]. Los resultados del perfilado se organizan en la Tabla 3.6.

Las capturas de perfilado obtenidas en el entorno de desarrollo (Figura 3.13 y Figura 3.14) confirman una reducción efectiva de $34,256$ Bytes en la IRAM [6]. Este espacio libre resultó indispensable para reservar el búfer dinámico necesario durante las tareas de actualización OTA.

3.5.2. Perfilado Térmico y Energético del Subclocking (80 MHz)

La reducción deliberada de la frecuencia del núcleo del SoC, desde el máximo nominal de 240 MHz hasta 80 MHz, se evaluó no solo como una medida de estabilidad funcional, sino como una decisión de gestión térmica y energética. La potencia dinámica disipada por un circuito CMOS se rige por la Ecuación 3.3:

$$P_{din} = C_{ef} \cdot V_{dd}^2 \cdot f \tag{3.3}$$

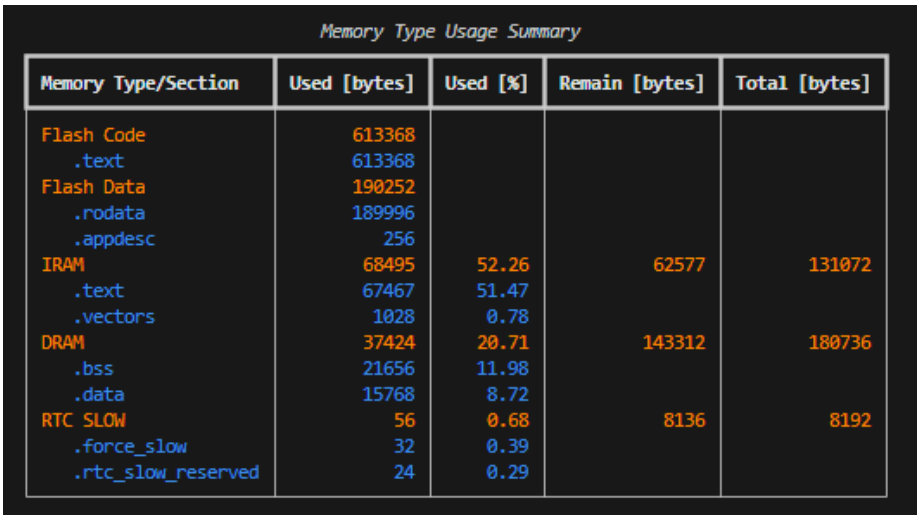
Tabla 3.6. Perfilado de recursos de memoria IRAM y DRAM en el SoC ESP32 ante optimizaciones de `menuconfig`.

Estado del Firmware	**IRAM Usada (Bytes)**	**IRAM (%)**	**DRAM Usada (Bytes)**	**Ahorro Directo**
1. Pila Completa (Wi-Fi IRAM Activo)	102,751	78.39 %	37,852	0.00 % (Crítico)
2. Reingeniería (Wi-Fi IRAM Inactivo)	68,495	52.26 %	37,424	**26.13 %**
Margen Libre Resultante	**62,577 B**	**47.74 %**	**143,312 B**	—

Memory Type Usage Summary

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	651014			
.text	651014			
Flash Data	199836			
.rodata	199580			
.appdesc	256			
IRAM	102751	78.39	28321	131072
.text	101723	77.61		
.vectors	1028	0.78		
DRAM	37852	20.94	142884	180736
.bss	21688	12.0		
.data	16164	8.94		
RTC SLOW	56	0.68	8136	8192
.force_slow	32	0.39		
.rtc_slow_reserved	24	0.29		

Figura 3.13. Perfilado de memoria inicial revelando ocupación crítica de IRAM (78.39 %).



Memory Type Usage Summary

Memory Type/Section	Used [bytes]	Used [%]	Remain [bytes]	Total [bytes]
Flash Code	613368			
.text	613368			
Flash Data	190252			
.rodata	189996			
.appdesc	256			
IRAM	68495	52.26	62577	131072
.text	67467	51.47		
.vectors	1028	0.78		
DRAM	37424	20.71	143312	180736
.bss	21656	11.98		
.data	15768	8.72		
RTC SLOW	56	0.68	8136	8192
.force_slow	32	0.39		
.rtc_slow_reserved	24	0.29		

Figura 3.14. Ocupación de IRAM optimizada al 52.26 % tras deshabilitar aceleración Wi-Fi.

donde C_{ef} es la capacitancia efectiva conmutada, V_{dd} la tensión de alimentación del núcleo y f la frecuencia de operación. Dado que en el ESP32 la tensión del núcleo es regulada internamente y permanece prácticamente constante, la frecuencia constituye la única variable de control directo sobre el consumo dinámico; en consecuencia, reducir f en un factor de 3 (de 240 a 80 MHz) contrae la potencia dinámica en la misma proporción. Esta reducción se traduce de forma directa en la disipación térmica, ya que el salto térmico entre la unión y el ambiente sigue la Ecuación 3.4:

$$\Delta T_{j-a} = P \cdot R_{\theta j-a} \tag{3.4}$$

con $R_{\theta j-a}$ como la resistencia térmica unión-ambiente, constante para un encapsulado y una placa dadas. Al reducirse P, el gradiente ΔT_{j-a} disminuye en la misma proporción, rebajando la temperatura de operación del silicio y, con ello, el estrés térmico acumulado en régimen continuo. La Tabla 3.7 sintetiza el compromiso entre frecuencia, potencia dinámica relativa y suficiencia criptográfica.

El aspecto crítico que habilita esta reducción es la presencia del acelerador criptográfico AES por hardware del ESP32: la operación intensiva de cifrado no se ejecuta en el núcleo, sino en un bloque dedicado, por lo que la frecuencia del CPU solo debe ser suficiente para alimentar y orquestar dicho acelerador. Bajo esta condición, el 80 MHz demostró sostener el handshake TLS en 45 ms, holgadamente dentro de la ventana de tolerancia de 100 ms (véase la Tabla 3.5). El efecto observable sobre la alimentación fue la estabilización de la

Tabla 3.7. Compromiso térmico-energético del subclocking del SoC ESP32.

Frecuencia del CPU	**Potencia Dinámica ($P \propto f$)**	**Salto Térmico ($\Delta T \propto P$)**	**Suficiencia Criptográfica (AES)**
240 MHz (nominal)	1.00 (referencia)	1.00 (referencia)	Sí, sobredimensionado
160 MHz	0.67	0.67	Sí
80 MHz (adoptado)	0.33	0.33	Sí (handshake TLS: 45 ms $<$ 100 ms)

corriente durante las ráfagas criptográficas: al no exigirse al núcleo picos de conmutación, la demanda instantánea se vuelve más uniforme, reduciendo las transitorias de corriente (di/dt) y la solicitación sobre el regulador de la placa. En términos de gestión térmica, la caída del salto térmico a un tercio del valor nominal se traduce en una operación más fría y en una menor degradación por ciclado térmico, elevando la decisión de subclocking del plano puramente funcional al del diseño térmico y energético del nodo.

3.5.3. Trazabilidad y Estabilidad de Almacenamiento en Servidor SQLite

En el nivel de nube se midió el crecimiento de la base de datos relacional SQLite (`db.sqlite3`) bajo ráfagas de telemetría continua [4]. La rutina de depuración automática del *Worker* (`fecha_limite` = `timezone.now()` − `timedelta(days=90)`) resultó efectiva: con un histórico simulado de $10,000$ registros, purga los obsoletos sin bloquear las consultas de la interfaz y mantiene la búsqueda indexada por serial por debajo de los 12 ms. La Tabla 3.8 resume las métricas obtenidas.

La trazabilidad se apoya en dos entidades: el dispositivo fiscal (identificado por su serial) y su bitácora (`LogDispositivo`), vinculada por clave foránea y ordenada cronológicamente. El *Worker* `listen_mqtt.py` compara cada trama con el último estado persistido y descompone la diferencia en eventos atómicos: cambios de `STS1`, transiciones de `STS2`, cortes de reporte Z, emisión de facturas, reconexiones y actualizaciones de firmware (véase el Código A.14), preservando la granularidad de la auditoría sin degradar la escritura.

La eficiencia se sostuvo en dos decisiones verificadas: la vista de auditoría emplea

Tabla 3.8. Métricas de trazabilidad y desempeño del almacenamiento en SQLite.

Métrica	**Valor medido**
Registros históricos de prueba	$10,000$
Latencia de búsqueda indexada por serial	< 12 ms
Ventana de retención automática	90 días
Estrategia de carga de relaciones	`select_related` + `prefetch_related`
Eventos registrados por trama	Uno o más, de forma atómica e independiente

`select_related` y `prefetch_related`, reduciendo las consultas N+1 a un número constante de accesos; y la búsqueda por serial usa coincidencia indexada (`serial__icontains`) con ordenamiento por fecha, manteniendo la latencia por debajo de los 12 ms incluso con el histórico completo. La retención de 90 días acota el volumen de forma continua, de modo que la profundidad de la bitácora permanece estable [4].

3.6. Cumplimiento de los Objetivos Específicos

Como cierre del capítulo y síntesis final del alcance del proyecto, la Tabla 3.9 contrasta los objetivos específicos planteados en la introducción con su grado de ejecución y el entregable técnico asociado. La matriz permite verificar la cobertura del objetivo general y deja constancia explícita de los desafíos superados, las optimizaciones aplicadas y los ajustes de alcance asumidos durante el desarrollo.

Tabla 3.9. Matriz de cumplimiento de objetivos específicos y alcance técnico.

Objetivo Específico	**Estado**	**Entregable / Resultado**	**Observaciones**
Diseñar la arquitectura IIoT que integre microcontroladores, protocolos y plataforma de monitoreo.	Logrado	Arquitectura de tres niveles (*Edge*, *Gateway* y *Cloud*) con diagramas de flujo y de comunicaciones.	Definición del árbol de tópicos y del modelo de seguridad (TLS y *User Properties*).
Desarrollar el firmware embebido para el microcontrolador ESP32.	Logrado	Firmware en C sobre ESP-IDF y FreeRTOS.	Reingeniería de la memoria IRAM (78.39 % → 52.26 %) y *subclocking* a 80 MHz.
Integrar el sistema con las interfaces del equipo fiscal para la captura de datos.	Logrado	Enlace síncrono SPI a 100 kHz y analizador de tramas *Status* S1/SV2.	Migración de UART a SPI por inmunidad electromagnética y corrección del error *Off-By-One*.
Implementar la comunicación mediante MQTT con un servidor externo.	Logrado	Cliente MQTT v5.0 sobre TLS 1.2 (puerto 8883).	Validación por *bundle* de certificados, *Last Will and Testament* y *User Properties*.
Implementar un servidor MQTT para recibir, almacenar y distribuir los datos.	Parcialmente Logrado	*Worker* MQTT en Django (Paho) con persistencia en SQLite.	El *broker* se delegó al servicio gestionado HiveMQ Cloud; se implementó la capa de recepción, almacenamiento y distribución.
Desarrollar una interfaz web que visualice en tiempo real el estado de los equipos.	Logrado	*Dashboard* Django/HTMX con cliente MQTT.js sobre WSS (puerto 8884).	Se descartó Django Channels para evitar un intermediario; integración directa navegador–*broker*.
Validar el sistema evaluando confiabilidad, latencia, frecuencia y robustez ante fallos.	Logrado	Banco de pruebas *Hardware-in-the-Loop* y métricas de latencia *end-to-end*.	Latencia media $T_{E2E} = 268$ ms y recuperación ante micro-cortes y alertas LWT.
Diseñar e implementar un módulo de actualización remota (OTA) de doble objetivo.	Logrado	Módulo OTA para el *gateway* (ESP32) y puente síncrono para el periférico fiscal.	Alcance ampliado respecto al plan original; reanudación de descargas por HTTP *Range* y máquina de estados finita.
Documentar el proceso de desarrollo y los resultados de validación.	Logrado	Diagramas, *scripts* de prueba y resultados de validación.	Código fuente y estructuras de mensajería conservados en el Apéndice A.

CONCLUSIONES

El desarrollo del presente proyecto culminó con el diseño, la implementación y la validación de un sistema IIoT de tres niveles (Edge, Gateway y Cloud) orientado a resolver una necesidad operativa concreta del parque de impresoras fiscales venezolano: la supervisión remota y el diagnóstico determinista de equipos que, hasta ahora, solo podían inspeccionarse mediante una conexión serial local. La solución fue validada en un entorno de laboratorio mediante un banco de pruebas Hardware-in-the-Loop, de modo que constituye un prototipo funcional y documentado, preparado para su despliegue en campo. Con todo, el trabajo sienta las bases técnicas para que The Factory HKA, empresa líder del sector, pueda centralizar la telemetría de un parque distribuido a nivel nacional y transformar la gestión reactiva del mantenimiento en una práctica basada en datos.

En el nivel de borde se demostró que es posible extraer el registro `Status S1` de forma transparente a través de la interfaz síncrona local, sin interferir con la máquina de estados legal del periférico ni interrumpir las transacciones fiscales en curso. Este resultado se sustentó en una re-ingeniería del kernel concurrente que eliminó las condiciones de carrera mediante exclusión mutua del bus físico y de la memoria compartida, y en una optimización de recursos que redujo la ocupación de IRAM del 78.39 % al 52.26 % mediante la reconfiguración de `menuconfig`, liberando el margen necesario para las operaciones de actualización. La combinación de determinismo temporal y eficiencia de memoria confirmó que el nodo de borde podía operar en un entorno industrial hostil sin comprometer la integridad del registro tributario.

La capa de red consolidó el ahorro operativo: la adopción de MQTT v5.0 sobre TLS 1.2 redujo el consumo de datos por trama en un 80.2 % frente a una arquitectura HTTP REST tradicional, al trasladar los metadatos a la cabecera del protocolo y aligerar el cuerpo del mensaje. A ello se sumó la eficacia del mecanismo *Last Will and Testament*, que permitió detectar de forma inmediata las desconexiones abruptas del borde y congelar el último estado válido, garantizando que una caída de energía o de enlace no se tradujera en una pérdida silenciosa de información fiscal.

En la nube, la plataforma construida sobre Django y HTMX demostró que es posible ofrecer una experiencia de aplicación de página única con un consumo de recursos mínimo, apoyada en un *worker* asíncrono que desacopla la ingesta de telemetría del servidor web. La persistencia en SQLite, organizada por eventos atómicos y con una ventana de retención de 90 días, otorgó al personal de soporte un historial de auditoría consultable por serial, cerrando la brecha de trazabilidad que la operación punto a punto no podía cubrir.

El componente más disruptivo fue el puente síncrono heterogéneo de reprogramación inalámbrica: el sistema es capaz de reescribir *in situ* tanto el firmware del nodo inteligente como el del periférico esclavo, mediante fragmentación de paquetes, verificación de *checksums* y reanudación ante cortes de red. Esta capacidad traslada el mantenimiento de firmware a la nube y reduce drásticamente la necesidad de intervención física en campo, un avance significativo para una empresa que administra equipos dispersos geográficamente.

En síntesis, la solución integra en una sola arquitectura el monitoreo, la trazabilidad y la actualización remota de equipos fiscales, y deja establecidas las condiciones técnicas para su despliegue en campo. El trabajo confirma la utilidad de un enfoque de ingeniería sustentado en el diagnóstico cuantitativo —perfilado de memoria, caracterización de latencias y pruebas de tolerancia a fallos— como método para resolver necesidades operativas reales en el ámbito de los sistemas embebidos y el Internet Industrial de las Cosas.

REFERENCIAS

[1] Amazon Web Services: *FreeRTOS Kernel Reference Manual*. Informe técnico, Amazon Web Services, 2024.

[2] Connected Everything: *Industrial Internet of Things Report: Industrial Systems in the Digital Age*. Informe técnico, Digital Catapult / IoTUK, 2016.

[3] Dierks, Tim y Rescorla, Eric: *The Transport Layer Security (TLS) Protocol Version 1.2*. RFC 5246, Internet Engineering Task Force, 2008.

[4] Django Software Foundation: *Django Web Framework Documentation v5.0*, 2026. `https://docs.djangoproject.com/`.

[5] Espressif Systems: *ESP-IDF Programming Guide v5.5.2*, 2025. `https://docs.espressif.com/projects/esp-idf/en/v5.5.2/esp32/get-started/`.

[6] Espressif Systems: *ESP-IDF Project Configuration Guide: SDK Configuration Tool (menuconfig)*, 2025. `https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/kconfig.html`.

[7] Espressif Systems: *ESP32 Series Datasheet*. Informe técnico, Espressif Systems, 2025. Versión 4.4.

[8] Fette, Ian y Melnikov, Alexey: *The WebSocket Protocol*. RFC 6455, Internet Engineering Task Force, 2011.

[9] Gross, Carson: *htmx: High-Power Tools for HTML*, 2024. `https://htmx.org/`.

[10] Mendoza, José Guillermo Fierro, Vera, Francisco Gutiérrez, España, Julio Armando Asato, González, Claudia Cristina Ortega y Juárez, Eduardo Alejandro Noria: *Análisis del uso de WebSockets para implementar aplicaciones web en tiempo real*. Pistas Educativas, 39(127), 2017, ISSN 2448-847X.

[11] National Institute of Standards and Technology: *Guidelines for the Selection, Configuration, and Use of Transport Layer Security (TLS) Implementations*. Informe técnico SP 800-52 Rev. 2, U.S. Department of Commerce, 2019.

[12] OASIS Open: *MQTT Version 5.0. OASIS Standard*, 2019. `https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html`.

[13] Salazar, Jordi y Silvestre, Santiago: *Internet de las Cosas*. České vysoké učení technické v Praze, TechPedia, 2014.

[14] The Factory HKA, C.A.: *Manual de Protocolos y Comandos Venezuela V0805R00-1*. Departamento de Soporte e Integración, 2022. Versión 8.5.0. Parámetros RS232 y estructuras S1/S2.

[15] The Factory HKA, C.A.: *Módulo de Transmisión iSmart-W: Manual de Especificaciones*, 2022.

[16] Vega, Pablo Alejandro Garrido: *Diseño e implementación de un sistema de monitoreo y visualización de temperatura en tiempo real para procesos de refrigeración*. Proyecto de Grado, Universidad Simón Bolívar, 2025.

[17] World Economic Forum: *Industrial Internet of Things Safety and Security Protocol*. Informe técnico REF 160418, Center for the Fourth Industrial Revolution, 2018.

APÉNDICE A

CÓDIGO FUENTE DEL SISTEMA

En este apartado se recopila una selección representativa de los fragmentos de código fuente más relevantes de los módulos desarrollados para el sistema de monitoreo remoto de equipos fiscales. Dado que la base de código del proyecto es considerablemente extensa, no resulta viable reproducirla en su totalidad dentro del informe; por ello, se incluyen únicamente las rutinas críticas que sustentan los resultados discutidos en el cuerpo del trabajo, y el código íntegro se conserva como anexo digital en el repositorio del proyecto. El material se organiza en cuatro bloques: el firmware del Módulo de Transmisión IoT (MT-IoT) en lenguaje C, las estructuras de mensajería en formato JSON, las herramientas de simulación y el *worker* de servidor en Python, y la capa de interfaz reactiva en HTML. Cada bloque conserva los comentarios originales que facilitan su interpretación por terceros.

A.1. Firmware del Módulo de Transmisión IoT (MT-IoT)

A.1.1. Sanitización y Parseo de la Trama SV2

Código A.1. Mitigación del error Off-By-One durante el parseo de la trama SV2.

```
int bytes_leidos = receive_response(rx_buffer, sizeof(rx_buffer), 1000);

if (bytes_leidos > 15 && validate_frame(rx_buffer, bytes_leidos)) {
    int etx_pos = bytes_leidos - 2; // Ajuste para esquivar salto de
        linea
    int fw_len = 10; // Ejemplo: 020507GD00

    if (etx_pos >= fw_len + 1) {
        int start_idx = etx_pos - fw_len - 1;
        if (xSemaphoreTake(mqtt_data_mutex, pdMS_TO_TICKS(100)) ==
            pdTRUE) {
```

```
            memset(mqtt_data.fw_printer, 0, sizeof(mqtt_data.fw_printer)
                );
            // Copiado seguro hacia la estructura
            strncpy(mqtt_data.fw_printer, (char*)&rx_buffer[start_idx],
                fw_len);
            mqtt_data.fw_printer[fw_len] = '\0'; // Terminador nulo
                manual

            ESP_LOGI(TAG_PRINTER, "Version␣detectada␣via␣SV2:␣%s",
                     mqtt_data.fw_printer);
            xSemaphoreGive(mqtt_data_mutex);
        }
        return true;
    }
}
```

A.1.2. Interrogación Serial No Bloqueante

Código A.2. Lógica no bloqueante de la tarea de interrogación serial (Polling).

```
void printer_polling_task(void *pvParameters) {
    uint8_t enq_cmd = 0x05; // Caracter ENQ segun el manual

    while(1) {
        // Transmision asincrona del byte de estado
        uart_write_bytes(UART_NUM_1, (const char*)&enq_cmd, 1);

        // El hilo cede voluntariamente el control al Scheduler por 5
            segundos
        vTaskDelay(pdMS_TO_TICKS(5000));
    }
}
```

A.1.3. Validación de Integridad de Trama

Código A.3. Lógica de validación de trama e intercepción de ruido electromagnético.

```
// Validar integridad estructural y matematica (LRC) de la
    trama
if (!validate_frame(rx_buffer, received)) {
```

```
    ESP_LOGW(TAG_PRINTER, "Trama␣invalida␣detectada␣por␣ruido
       /Checksum");
    printer_handle_error(ERROR_INVALID_RESPONSE);
    // Liberacion del semaforo para evitar bloqueos del
       sistema
    xSemaphoreGive(spi_bus_mutex);
    continue;
}
```

A.1.4. Exclusión Mutua de Memoria y Bus Físico

Código A.4. Uso de doble exclusión mutua para proteger RAM y hardware físico.

```
// 1. Tomar el control exclusivo del cable fisico sincrono
if (xSemaphoreTake(bus_local_mutex, portMAX_DELAY) == pdTRUE)
    {

    // Validar integridad estructural y matematica de la
       trama
    if (validate_frame(rx_buffer, received)) {

        // 2. Tomar el control de la memoria RAM para
           escritura
        if (xSemaphoreTake(mqtt_data_mutex, pdMS_TO_TICKS
           (100)) == pdTRUE) {
            strncpy(mqtt_data.fw_printer, (char*)&rx_buffer[
               start_idx], fw_len);
            mqtt_data.fw_printer[fw_len] = '\0';
            xSemaphoreGive(mqtt_data_mutex); // Liberar RAM
        }
    }
    xSemaphoreGive(bus_local_mutex); // Liberar cable fisico
}
```

A.1.5. Resiliencia y Reconexión MQTT

Código A.5. Lógica de resiliencia y reconexión en la tarea MQTT.

```
// VERIFICACION DE CONEXION DENTRO DE LA TAREA ASINCRONA
if (!mqtt_connected) {
    LOG_W(TAG_MQTT, "MQTT␣desconectado,␣reintentando␣en␣breve
       ...");
    // Delay de cortesia para no saturar la red (Retardo
       exponencial)
    vTaskDelay(pdMS_TO_TICKS(RECONNECT_DELAY_MS));
    continue;
}
```

A.1.6. Configuración Segura del Cliente MQTT

Código A.6. Configuración de inicialización del cliente MQTT con seguridad TLS estricta.

```
esp_mqtt_client_config_t mqtt_cfg = {
    .broker.address.hostname = "832
       b8689599f4045be005c116bc416f0.s1.eu.hivemq.cloud",
    .broker.address.port = 8883,
    .broker.address.transport = MQTT_TRANSPORT_OVER_SSL,
    .broker.verification.crt_bundle_attach =
       esp_crt_bundle_attach,
    // ... configuracion de credenciales y sesion ...
};
```

A.1.7. Inyección de Metadatos mediante User Properties

Código A.7. Inyección dinámica de metadatos mediante User Properties de MQTT v5.0.

```
esp_mqtt5_user_property_item_t user_prop_items[] = {
    {"modelo_dispositivo", "HKA80"},
    {"firmware_version", ISMART_VERSION},
```

```
    {"serial", serial_seguro},
    {"tipo_dato", tipo_dato},
    {"wifi_rssi", rssi_str} // Capturado via
       esp_wifi_sta_get_ap_info
};

mqtt5_user_property_handle_t user_prop_handle = NULL;
esp_mqtt5_client_set_user_property(&user_prop_handle,
   user_prop_items, 5);
```

A.1.8. Testamento (Last Will and Testament)

Código A.8. Mensaje de última voluntad (*Last Will and Testament*) registrado en el broker.

```
{"st": "offline", "alerta": "conexion_perdida_abruptamente"}
```

A.1.9. Reanudación de Descargas mediante HTTP Range

Código A.9. Inyección de cabecera HTTP Range para reanudación de descargas interrumpidas.

```
if (total_bytes_recibidos > 0) {
    char range_header[32];
    // Solicitamos al servidor continuar desde el byte de
       corte exacto
    snprintf(range_header, sizeof(range_header), "bytes=%d-",
        total_bytes_recibidos);
    esp_http_client_set_header(client, "Range", range_header)
       ;
}
esp_err_t err = esp_http_client_open(client, 0);
```

A.1.10. Empaquetado de Firmware Segmentado hacia el Periférico

Código A.10. Empaquetado y transmisión de firmware segmentado hacia el periférico local.

```
// Empaquetado estricto de capa fisica
transaccionar_byte_bus(CMD_PACKET); // Comando de insercion
enviar_chunk_bus((uint8_t*)&num_pack, 4); // Secuencial del
   paquete
enviar_chunk_bus((uint8_t*)&total_packs, 4); // Total
   esperado
uint16_t tam_actual = (uint16_t)len;
enviar_chunk_bus((uint8_t*)&tam_actual, 2); // Longitud del
   fragmento
enviar_chunk_bus(data, len); // Bloque de datos crudos
enviar_chunk_bus((uint8_t*)&checksum_paquete, 4); //
   Redundancia

if (transaccionar_byte_bus(0x00) == ACK_BYTE) {
    return ESP_OK; // Bloque integrado exitosamente
}
```

A.2. Estructuras de Mensajería (JSON)

A.2.1. Telemetría del Status S1

Código A.11. Estructura JSON del Status S1 publicado por el MT-IoT hacia el broker MQTT.

```
{
  "timestamp": 164370,
  "sts1": "0x60",
  "sts2": "0x40",
  "estado": "Modo Fiscal y en Espera",
  "error": "Ningun error",
  "cajero": "0010",
  "ventas": "00000000000000000",
  "fact_emitidas": "00000",
  "rif": "J3121711970",
  "registro": "Z1F9999988"
}
```

A.2.2. Comando de Actualización Remota

Código A.12. Estructura del payload MQTT para iniciar un proceso de actualización remota.

```
{
  "comando": "INICIAR_OTA",
  "objetivo": "ismart",
  "version": "v020100",
  "url_descarga": "https://servidor-seguro.com/media/firmwares/update.
     bin"
}
```

A.3. Herramientas de Simulación

Código A.13. Simulador fiscal con cálculo de redundancia (LRC) y manejo de estados.

```
def procesar_trama_completa(ser, trama):
    # Extraccion de datos y byte de comprobacion LRC
    data_etx = trama[1:-1]
    lrc_rx = trama[-1]
    lrc_calc = calcular_lrc(data_etx)

    if lrc_rx == lrc_calc:
        ser.write(bytes([0x06])) # Enviar ACK (0x06)
        comando = data_etx[:-1].decode('utf-8', errors='ignore')

        if comando == "S1":
            enviar_respuesta_s1(ser) # Trama con variables fiscales
        elif comando == "SV2":
            enviar_respuesta_sv2(ser) # Version de firmware
    else:
        ser.write(bytes([0x15])) # Enviar NAK por error de LRC
```

A.4. Worker de Telemetría en el Servidor

Código A.14. Lógica de suscripción y manejo de desconexiones LWT en el Worker MQTT.

```
def on_message(client, userdata, msg):
```

```
    partes_topico = msg.topic.split('/')
    serial = partes_topico[2]
    tipo = partes_topico[3] # ej. 'alertas_conexion' o 'json_completo'

    payload = json.loads(msg.payload.decode('utf-8'))

    # Interceptacion de Alertas de Red (Testamento LWT)
    if tipo == "alertas_conexion" and payload.get("st") == "offline":
        # Clonacion transaccional del ultimo log valido para no perder
            el estado
        dispositivo = Dispositivo.objects.get(serial=serial)
        LogDispositivo.objects.create(
            dispositivo=dispositivo,
            evento="Desconexion␣Abrupta␣(LWT)",
            detalles=payload
        )
```

A.5. Interfaz Reactiva Web

Código A.15. Captura reactiva de eventos MQTT mediante atributos declarativos de HTMX.

```
<!-- log_capture.html -->
<!-- hx-trigger escucha el evento despachado por el cliente WebSockets (
    MQTT.js) -->
<div id="vista-log-capture" class="p-4"
    hx-get="/log_capture/?buscar={{␣request.GET.buscar␣}}"
    hx-trigger="mqttNuevoLog␣from:body"
    hx-target="#vista-log-capture"
    hx-swap="outerHTML">

    <div class="d-flex␣justify-content-between␣align-items-center␣mb-4">
        <h4>Historial de Auditoria por Dispositivo</h4>
        <small class="text-muted">Conservacion automatica: 90 dias</
            small>
    </div>
    <!-- ... renderizado dinamico de la tabla ... -->
</div>
```